

Architectural Styles and Patterns

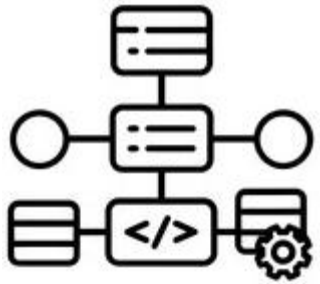
Estilos y Patrones Arquitectonicos

Universidad de La Sabana

FACULTAD DE INGENIERÍA

Universidad de La Sabana
FACULTAD DE INGENIERIA

Agenda



Architectural Styles and Patterns

- Why
  - Software Development Lifecycle
  - Business Driver/Motivator
- What/What Extend
  - Software Requirements
    - Gathering — QAW
    - ASR
  - Quality Attributes
    - Availability
    - Performance
    - .....
  - Tactics
    - Heartbeat
    - Voting
    - Retry
    - Increase Resources
    - .....
  - Architectural Patterns
    - Rate Limiting
    - Sidecar
    - CQRS
    - .....
  - Architectural Styles
    - Layered
    - Modular Monolith
    - EDA
    - Microservices
    - .....
- How
  - Attribute-Driven Design - ADD

SDLC and SW Req

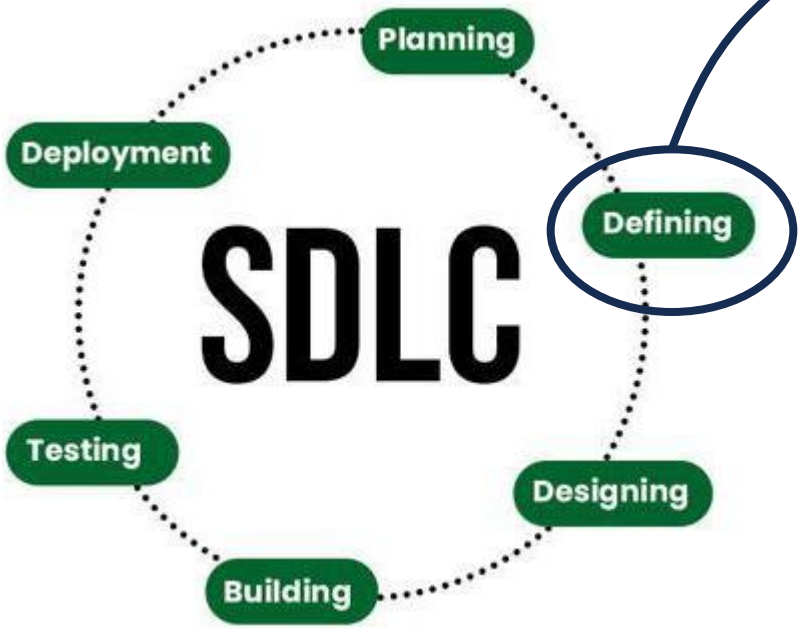


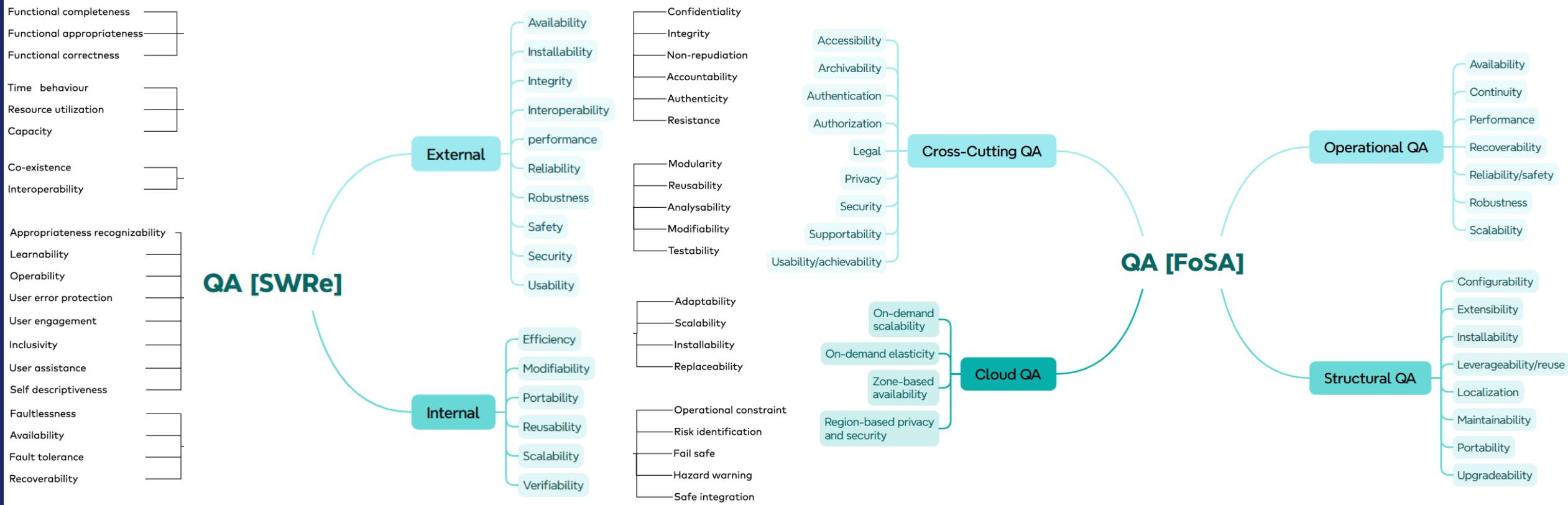
TABLE 1-1 Some types of requirements information

Term	Definition
Business requirement	A high-level business objective of the organization that builds a product or of a customer who procures it.
Business rule	A policy, guideline, standard, or regulation that defines or constrains some aspect of the business. Not a software requirement in itself, but the origin of several types of software requirements.
Constraint	A restriction that is imposed on the choices available to the developer for the design and construction of a product.
External interface requirement	A description of a connection between a software system and a user, another software system, or a hardware device.
Feature	One or more logically related system capabilities that provide value to a user and are described by a set of functional requirements.
Functional requirement	A description of a behavior that a system will exhibit under specific conditions
Nonfunctional requirement	A description of a property or characteristic that a system must exhibit or a constraint that it must respect.
Quality attribute	A kind of nonfunctional requirement that describes a service or performance characteristic of a product.
System requirement	A top-level requirement for a product that contains multiple subsystems, which could be all software or software and hardware.
User requirement	A goal or task that specific classes of users must be able to perform with a system, or a desired product attribute.

There are a different names for the same property: {System} Quality Attribute [SAiP, SWRe], Architectural Characteristics [FoSA], Capabilities (of the System) [HFSoA], “-illities”, cross-cutting requirements, capabilities….

Universidad de La Sabana
FACULTAD DE INGENIERIA

Arch. Characteristics/Quality Attributes (QA)



Even thought those sources and classification, there are ambiguous definition, i.e. *learnability* (users or system?) or for elements, TCP/IP (*available vs reliable*) [SoFA].

Universidad de La Sabana
FACULTAD DE INGENIERIA

Quality Attributes (QA) and its scope

Table 14-5. Translating quality attributes into technical specifications

Quality attributes	Likely technical information category
Installability, integrity, interoperability, reliability, robustness, safety, security, usability, verifiability	Functional requirement
Availability, efficiency, modifiability, performance, reliability, scalability	System architecture
Interoperability, security, usability	Design constraint
Efficiency, modifiability, portability, reliability, reusability, scalability, verifiability, usability	Design guideline
Portability	Implementation constraint

No ubiquitous language.
Over-engineering !

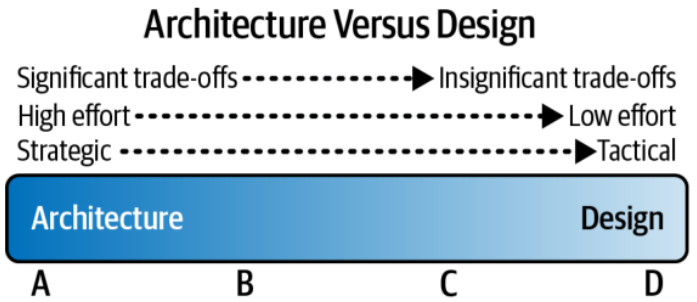


Figure 2-1. The spectrum between architecture and design

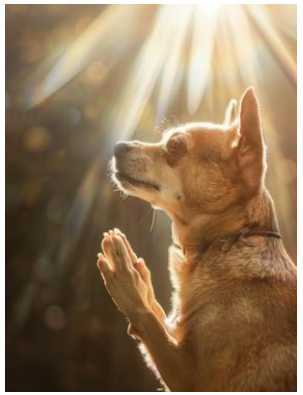
TABLE 14-1 Some software quality attributes

External quality	Brief description
Availability	The extent to which the system's services are available when and where they are needed
Installability	How easy it is to correctly install, uninstall, and reinstall the application
Integrity	The extent to which the system protects against data inaccuracy and loss
Interoperability	How easily the system can interconnect and exchange data with other systems or components
Performance	How quickly and predictably the system responds to user inputs or other events
Reliability	How long the system runs before experiencing a failure
Robustness	How well the system responds to unexpected operating conditions
Safety	How well the system protects against injury or damage
Security	How well the system protects against unauthorized access to the application and its data
Usability	How easy it is for people to learn, remember, and use the system
Internal quality	**Brief description**
Efficiency	How efficiently the system uses computer resources
Modifiability	How easy it is to maintain, change, enhance, and restructure the system
Portability	How easily the system can be made to work in other operating environments
Reusability	To what extent components can be used in other systems
Scalability	How easily the system can grow to handle more users, transactions, servers, or other extensions
Verifiability	How readily developers and testers can confirm that the software was implemented correctly

TIP

Never strive for the *best* architecture; aim for the *least worst* architecture.

Common vocabulary
[Doing at your org!]



[FoSA] and [SWRe]

Universidad de La Sabana
FACULTAD DE INGENIERIA

Explicit or Implicit Definition

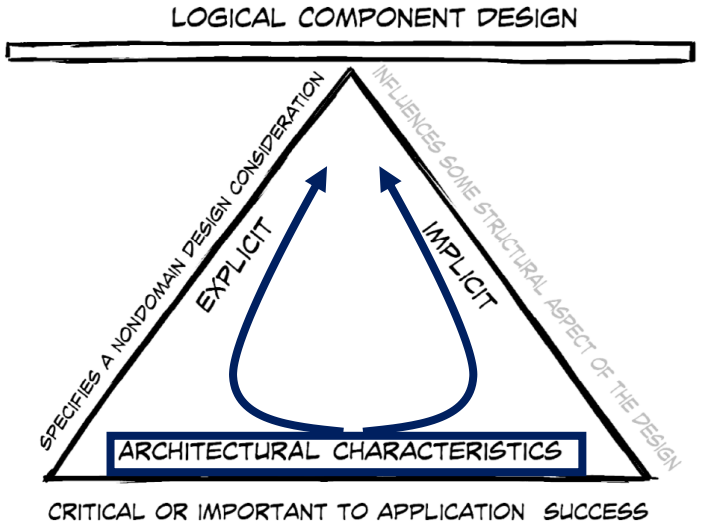


Table 5-1. Translating domain concerns into architectural characteristics

Domain concern	Architectural characteristics
Mergers and acquisitions	Interoperability, scalability, adaptability, extensibility
Time to market	Agility, testability, deployability
User satisfaction	Performance, availability, fault tolerance, testability, deployability, agility, security
Competitive advantage	Agility, testability, deployability, scalability, availability, fault tolerance
Time and budget	Simplicity, feasibility

⇩

Low-Code or No-Code Platforms

ORACLE APPLICATION EXPRESS

AppSheet + Google Workspace

AWS Amplify

Microsoft PowerApps

Over-engineering !

What are the priorities ?

There are no wrong answers in architecture, only expensive ones.

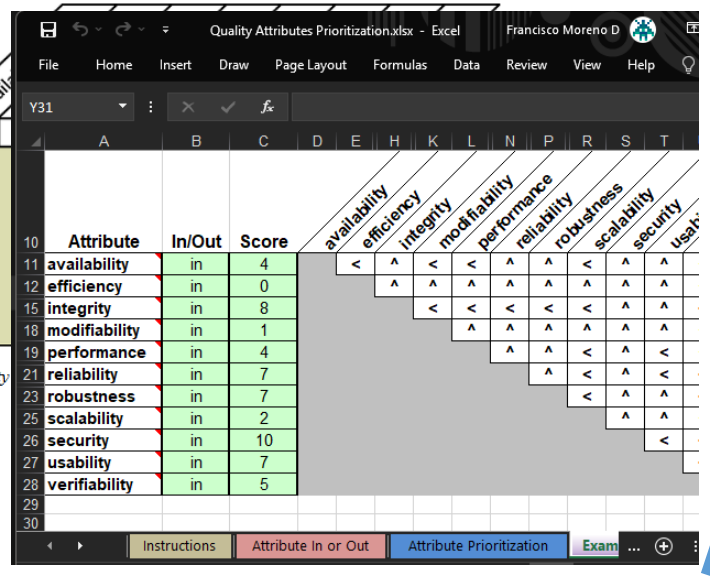
—One of Mark's famous quotes

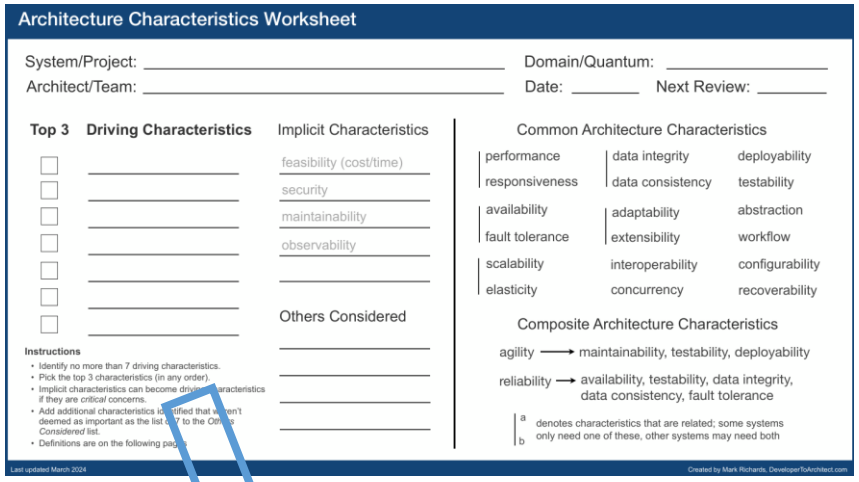
Universidad de La Sabana
FACULTAD DE INGENIERÍA

Prioritize attributes

Attribute	Score
Availability	2
Integrity	6
Performance	4
Reliability	2
Robustness	1
Security	7
Usability	5
Verifiability	1

Figure 14-1. Sample quality





Steps to prioritize [SWRe]:

1. Start with a broad taxonomy
2. Reduce the list
3. Prioritize the attributes: Spreadsheet or worksheet
4. Elicit specific expectations for each attribute
5. Specify well-structured quality requirements

Take care of this synergy ?

	Availability	Efficiency	Installability	Integrity	Interoperability	Modifiability	Performance	Portability	Reliability	Reusability	Robustness	Safety	Scalability	Security	Usability	Verifiability
Availability								+		+						
Efficiency	+				−	−	+	−			−		+		−	
Installability	+							+					+			
Integrity			−		−		−			−		+		+	−	−
Interoperability	+		−	−			−	+	+		+	−		−		
Modifiability	+		−				−		+	+			+			+
Performance		+			−	−		−			−		−		−	
Portability		−			+	−	−			+				−	−	+
Reliability	+	−		+		+	−				+	+		+	+	+
Reusability		−		−	+	+	−	+						−		+
Robustness	+	−	+	+	+		−		+			+	+	+	+	
Safety		−		+	+		−				+			+	−	−
Scalability	+	+		+			+	+	+		+					
Security	+			+	+		−	−	+		+	+			−	−
Usability		−	+				−	−	+		+	+				−
Verifiability	+		+	+		+			+	+	+	+		+	+	

Figure 14-2. Positive and negative relationships among selected quality attributes.

Quality Attribute Workshop (QAW)

Carnegie Mellon University

Software Engineering Institute About Our Worl

Home › Publications › Digital Library › Quality Attribute Workshop Collection

Quality Attribute Workshop Collection



3 QAW Method 7
3.1 Step 1: QAW Presentation and Introductions 8
3.2 Step 2: Business/Mission Presentation 8
3.3 Step 3: Architectural Plan Presentation 8
3.4 Step 4: Identification of Architectural Drivers 9
3.5 Step 5: Scenario Brainstorming 9
3.6 Step 6: Scenario Consolidation 10
3.7 Step 7: Scenario Prioritization 10
3.8 Step 8: Scenario Refinement 11

4 QAW Benefits 13

Thumb Rules:

- Play the dumb: *obtain the signature*
- Assumptions and mentioned/writing on presentations. Make explicit.

Other Options to Deep:

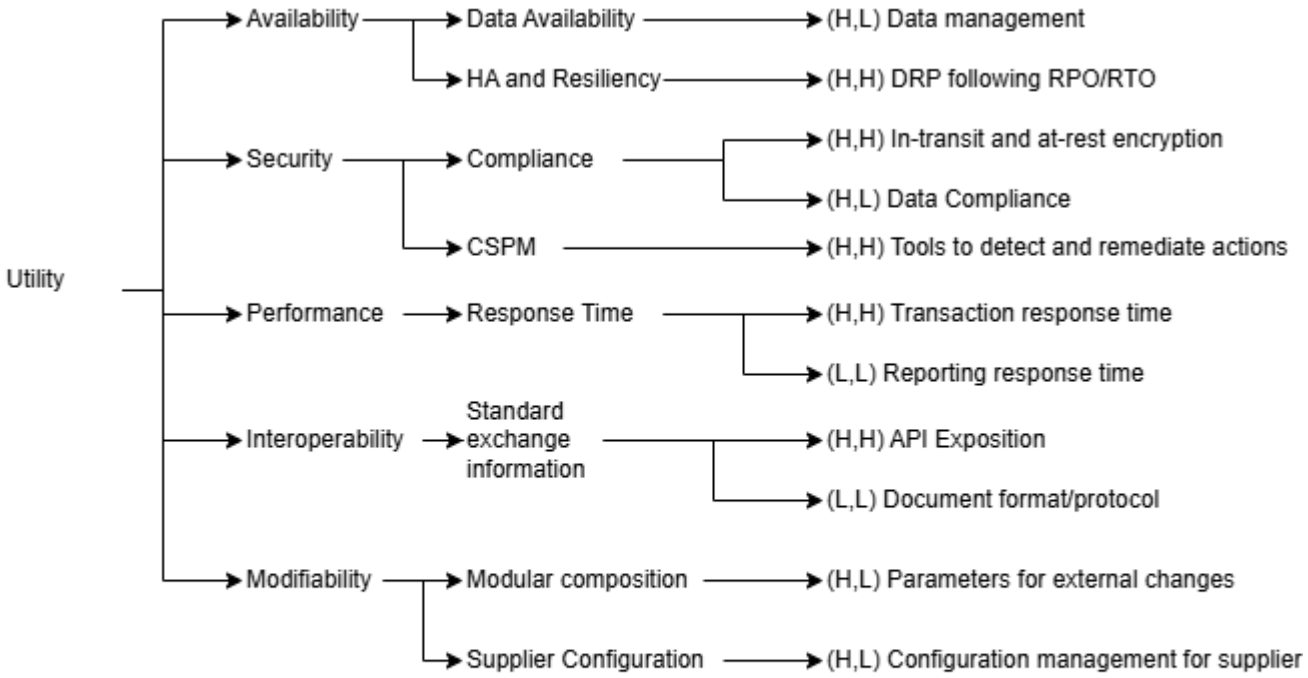
- PALM, Pedigreed Attribute eLicitation Method
- TOGAF, Business Architecture

Link

Utility Tree

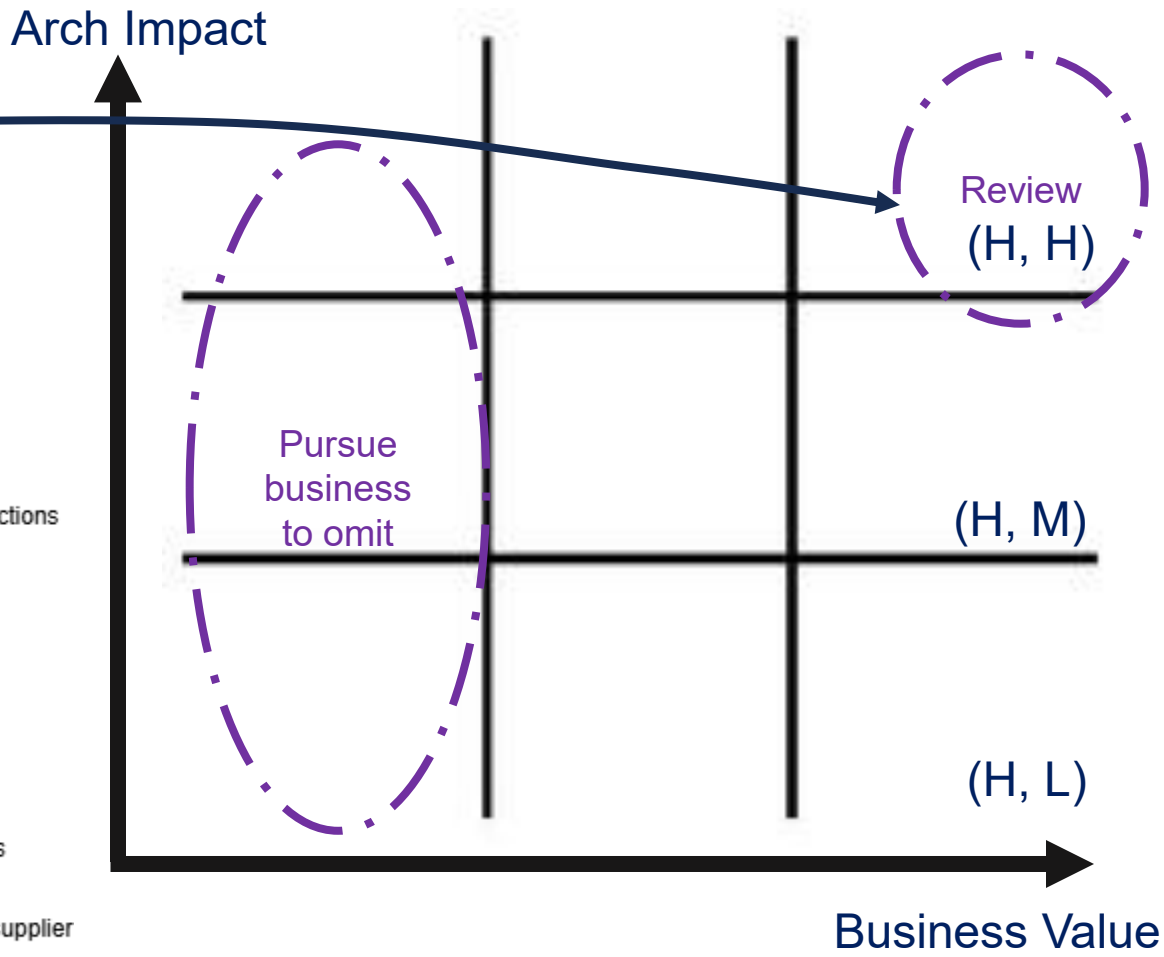
TABLE 19.1 Tabular Form of the Utility Tree for a System in the Healthcare Space

Quality Attribute	Attribute Refinement	ASR Scenario
Performance	Transaction response time	A user updates a patient's account in response to a change-of-address notification while the system is under peak load, and the transaction completes in less than 0.75 seconds. (H, H)
	Throughput	At peak load, the system is able to complete 150 normalized transactions per second. (M, M)



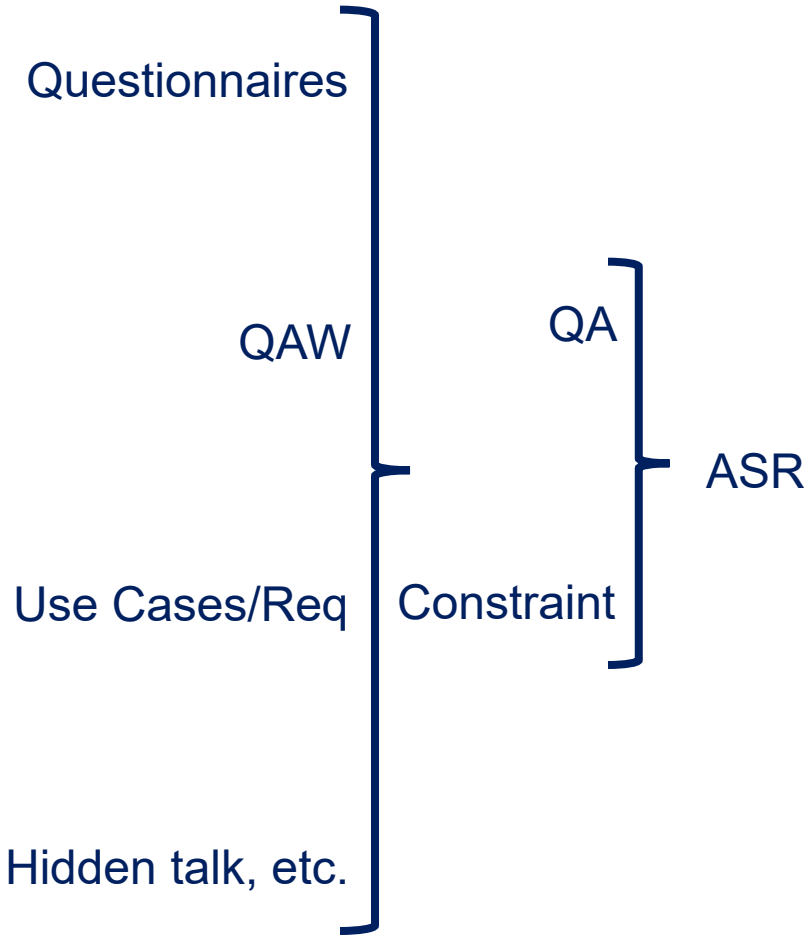
Legend

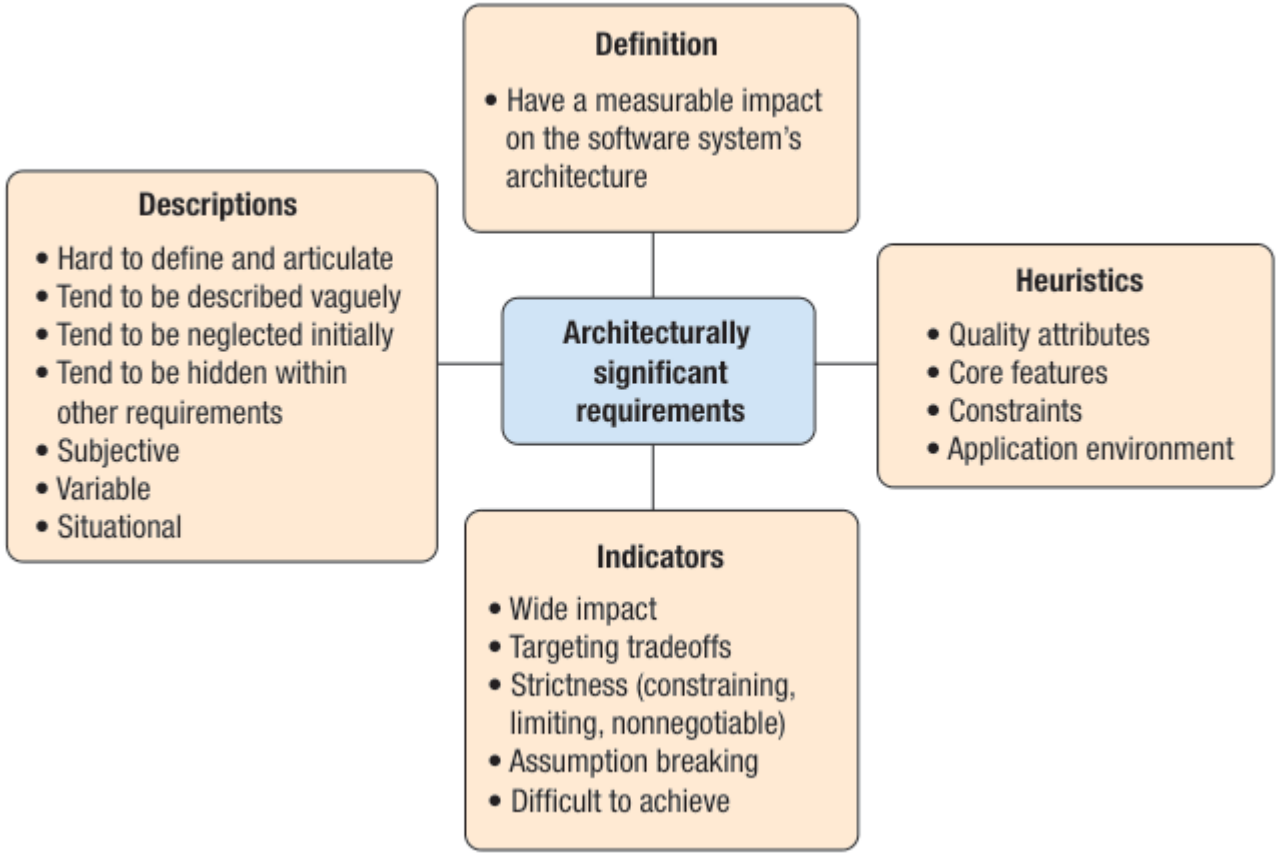
(X, Y) X= Business Value, Y=Architecture Impact



Architecturally Significant Requirement (ASR)

"requirement that will have a profound effect on the architecture—that is, the architecture might well be dramatically different in the absence of such a requirement"





Measure Quality Attributes (QA)

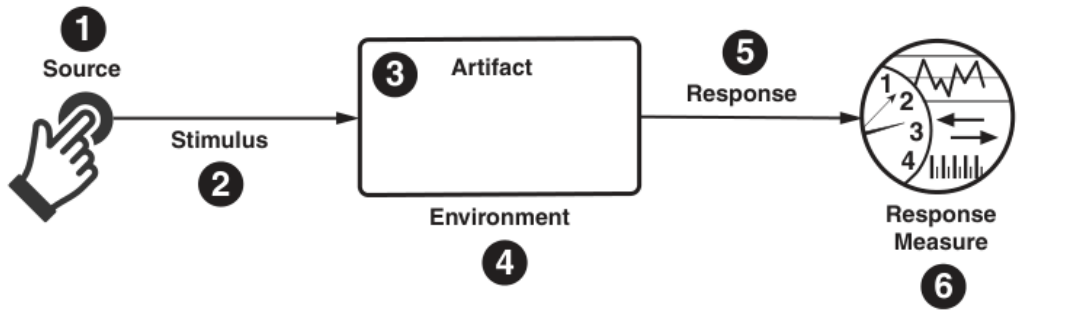


FIGURE 3.1 The parts of a quality attribute scenario

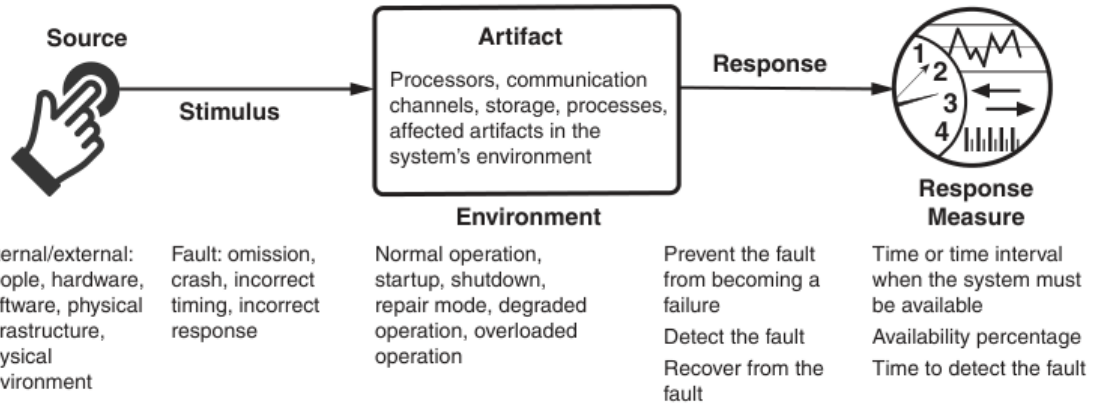


FIGURE 3.2 A general scenario for availability

"Quality attribute requirements are satisfied by the structures and behaviors of the architecture." [SAiP, C3.7]

No manner to measure an attribute directly, instead we create QA scenarios to measure its properties. Take care of synergy.

Stimulus source: i.e. *AuthN or anonymous user.*
Stimulus: Event arriving at system (i.e. *event, user op, attack*).
Response: Activity that occurred as result of stimulus.
Response measure: How we can measure/achieve?
Environment: Set the context of the scenario (circumstances, i.e. *runtime, not-runtime, 5th successive failure*).
Artifact: Specific component where stimulus arrive.

Universidad de La Sabana
FACULTAD DE INGENIERIA

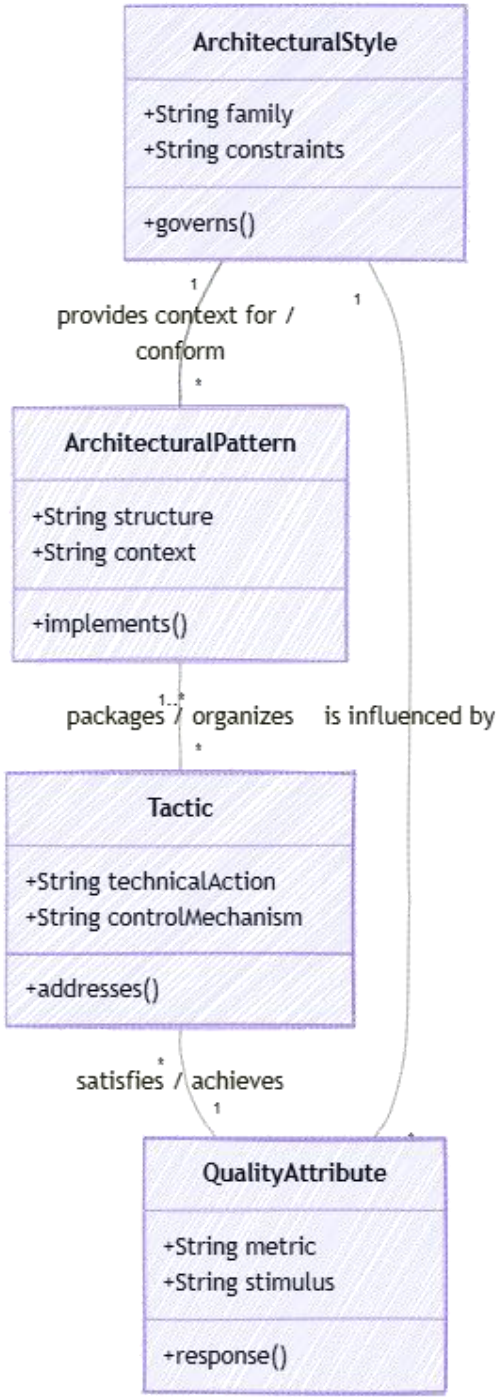
Achieve QA through arch patterns and tactics

"A tactic is a design decision that influences the achievement of a QA response (..). An architectural pattern describes a particular recurring design problem that arises in specific design contexts and presents a well-proven architectural solution for the problem".

" Like the choice of tactics, the choice of an architectural pattern has a profound effect on QA—usually more than one.
(...)
Patterns typically comprise multiple design decisions and, in fact, often comprise multiple tactics. We say that patterns often bundle tactics and, consequently, frequently make tradeoffs among quality attributes."

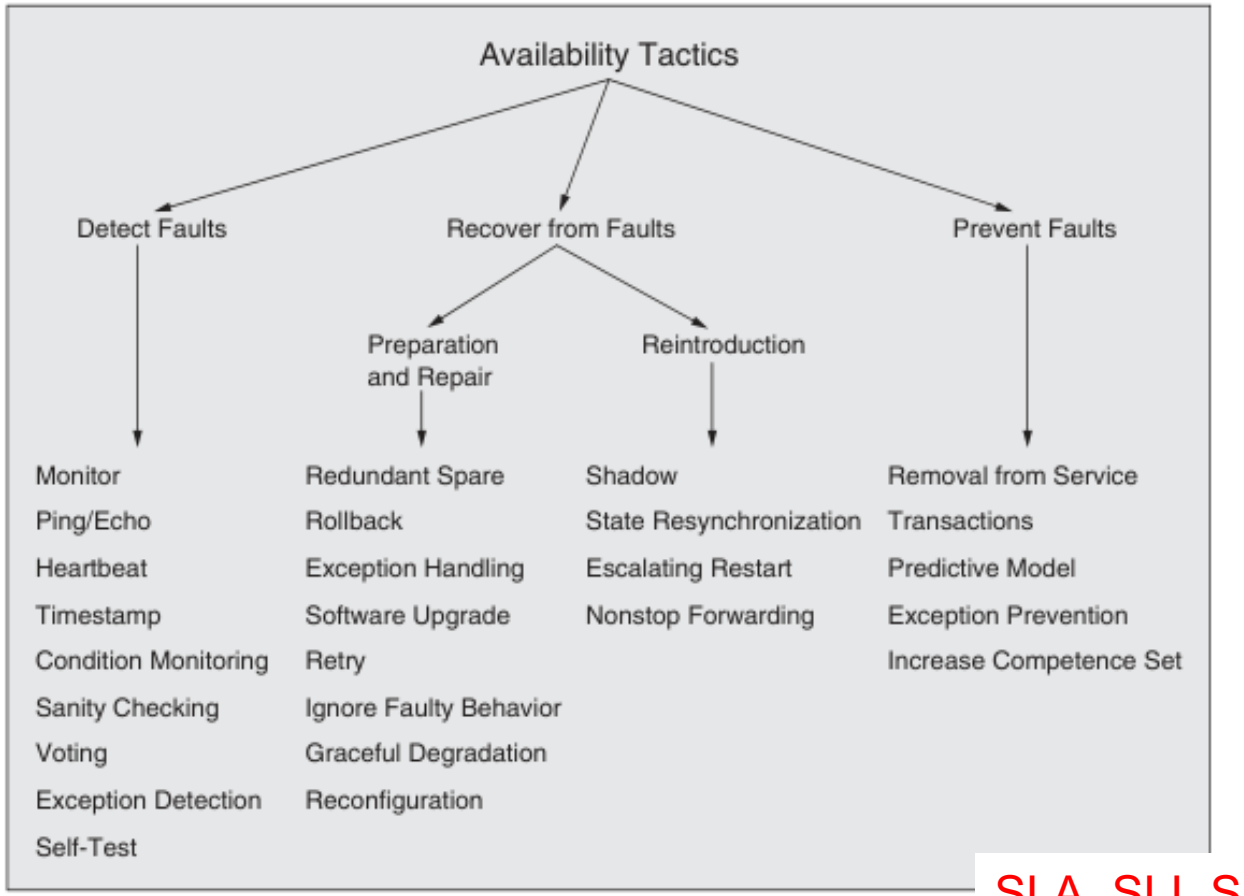
"While we discuss patterns and tactics as though they were foundational design decisions, the reality is that architectures often emerge and evolve as a result of many small decisions and business forces". **>> Architecture Decay**

"Successfully achieving quality attributes often involves process-related decisions, in addition to architecture-related decisions". **>> Human Factor**



Availability Tactics





SLA, SLI, SLO

FIGURE 4.3 Availability tactics

"that **it is there (place)** and ready to **carry out its task (order)** when you need it to be" [SAiP], "the proportion of time that the system is functional and working" [MsAAG]

- Reliability: Add by "recovery" concept.

Linked with:

- Security: If there are hacked, i.e. ransomware. No available until payment.
- Performance: Available, but the degraded time, customer leaves.

Failure (deviation from specification) << Fault (cause)

TABLE 4.1 System Availability Requirements

Availability	Downtime/90 Days	Downtime/Year
99.0%	21 hr, 36 min	3 days, 15.6 hr
99.9%	2 hr, 10 min	8 hr, 0 min, 46 sec
99.99%	12 min, 58 sec	52 min, 34 sec
99.999%	1 min, 18 sec	5 min, 15 sec
99.9999%	8 sec	32 sec

[SAiP]

Universidad de La Sabana
FACULTAD DE INGENIERIA

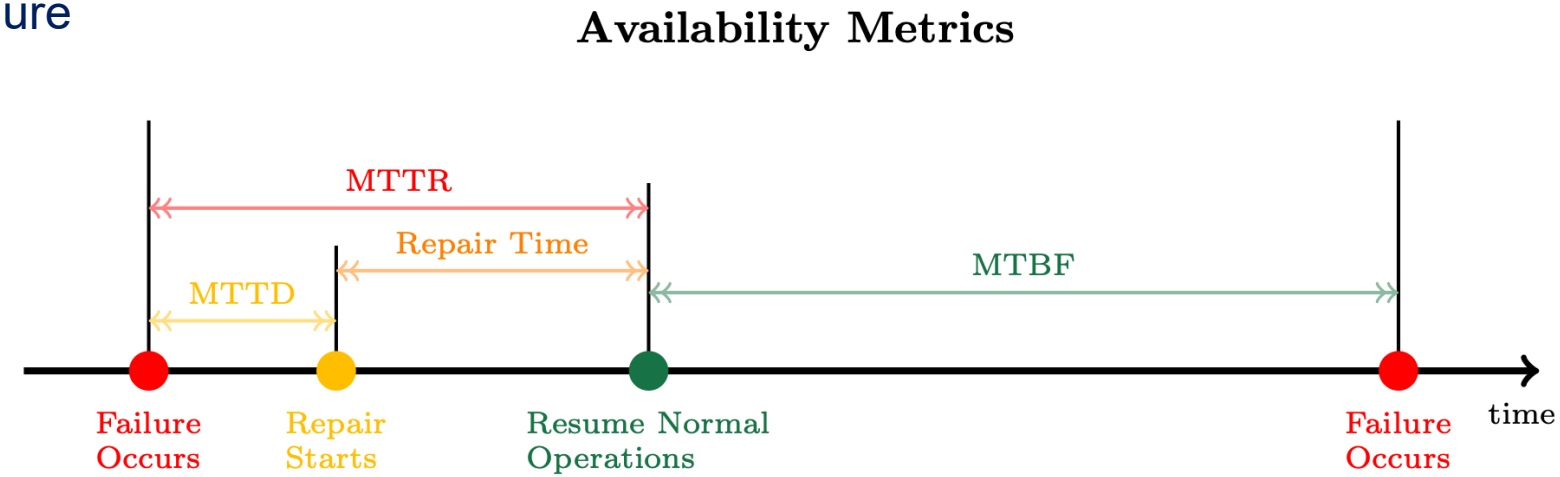
Availability Metrics

$$A = \frac{uptime}{uptime + downtime}$$

$$A = \frac{MTBF}{MTBF + MTTR}$$

$$F = 1 - A$$

A = % Uptime
F = Prob. Failure



Availability with dependencies (Sequence)

$$A = \alpha_1 \times \alpha_2 \times \ldots \times \alpha_n$$

Availability with redundancy (Parallel)

$$A = 1 - F$$
$$F = (1 - \alpha_1) \times (1 - \alpha_2)$$
$$A = 1 - (1 - \alpha)^2$$

mean time to detection (MTTD)
mean time to repair (or recovery) (MTTR)
mean time between failure (MTBF)

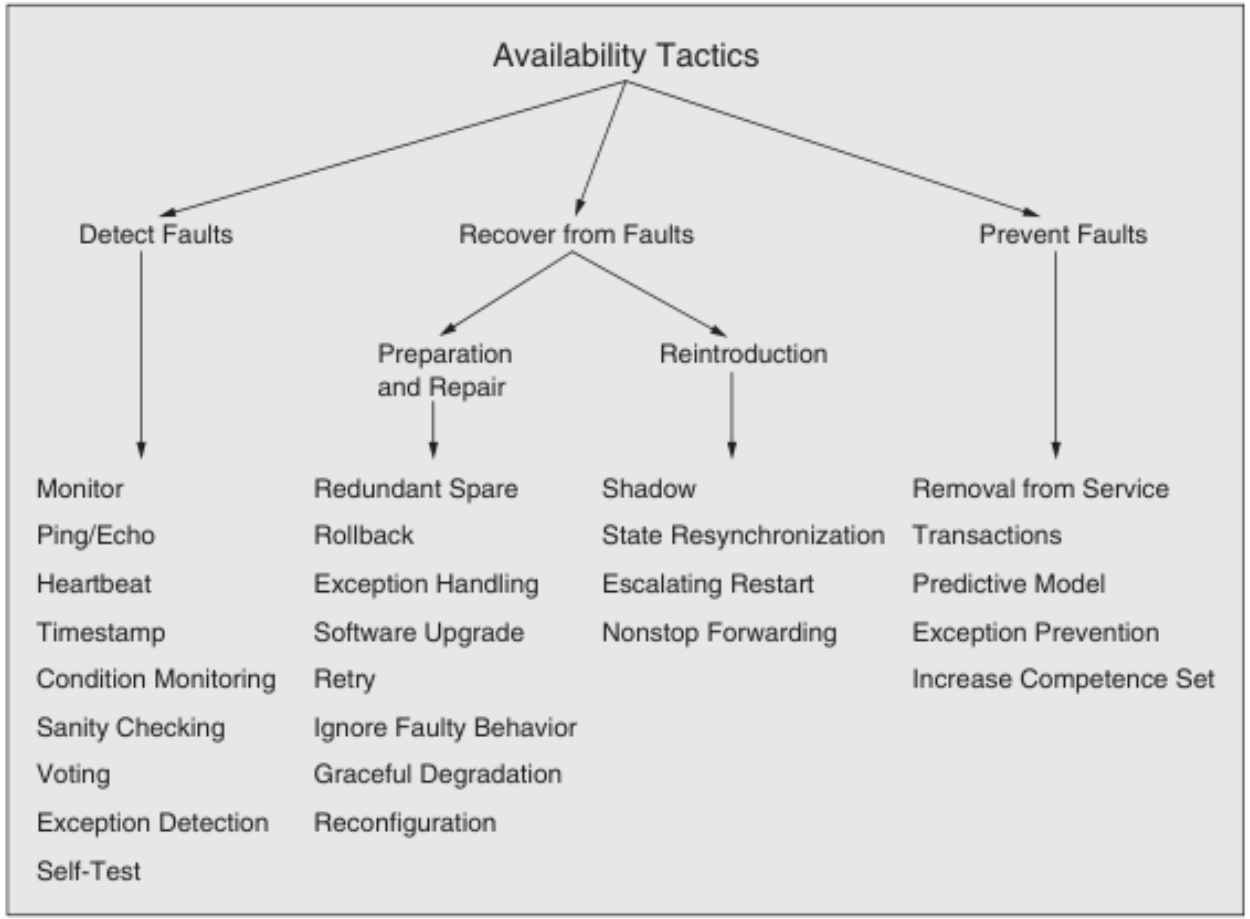
Universidad de La Sabana
FACULTAD DE INGENIERIA

Availability Patterns

Measurements

Response measure	We may focus on a number of measures of availability, depending on the criticality of the service being provided.	▪ Time or time interval when the system must be available ▪ Availability percentage (e.g., 99.999 percent) ▪ Time to detect the fault ▪ Time to repair the fault ▪ Time or time interval in which system can be in degraded mode ▪ Proportion (e.g., 99 percent) or rate (e.g., up to 100 per second) of a certain class of faults that the system prevents, or handles without failing

Tactics



Patterns

- Active redundancy (hot spare)
- Passive redundancy (warm spare)
- Spare (cold spare)
- Triple modular redundancy (TMR)
- Circuit breaker
- Process pairs
- Forward error recovery

Performance Tactics



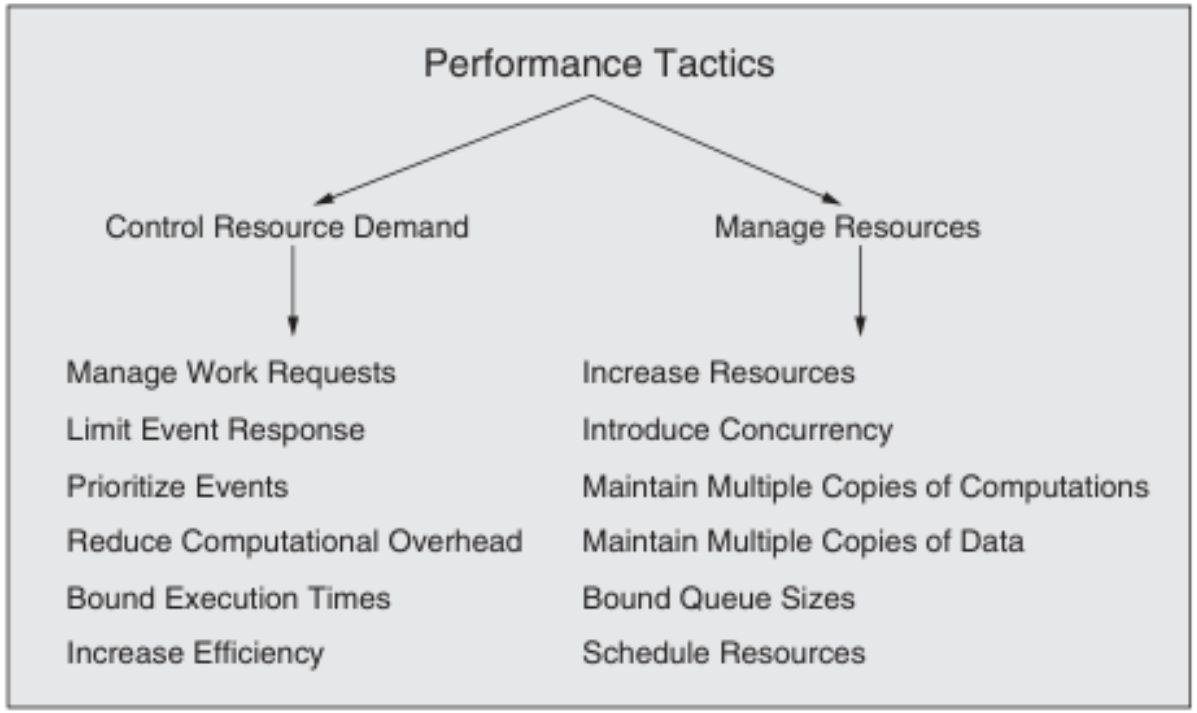


FIGURE 9.3 Performance tactics

"It's about **time** and (..) ability to meet timing **requirements**" [SAiP],
"responsiveness of a system to execute specific actions in a given **time interval**" [MsAAG]

- Scalability: Influenced by number of requests.

Linked with:
- Response Time
- Latency (even *Jitter*)
- Throughput
- Bandwidth

Tactics looks more contention and scalability

[SAiP]

Universidad de La Sabana
FACULTAD DE INGENIERIA

Performance Patterns

Measurements

Response measure	Timing measures can include latency or throughput. Systems with timing deadlines can also measure jitter of response and ability to meet the deadlines. Measuring how many of the requests go unsatisfied is also a type of measure, as is how much of a computing resource (e.g., a CPU, memory, thread pool, buffer) is utilized.	▪ The (maximum, minimum, mean, median) time the response takes (latency) ▪ The number or percentage of satisfied requests over some time interval (throughput) or set of events received ▪ The number or percentage of requests that go unsatisfied ▪ The variation in response time (jitter) ▪ Usage level of a computing resource

Tactics

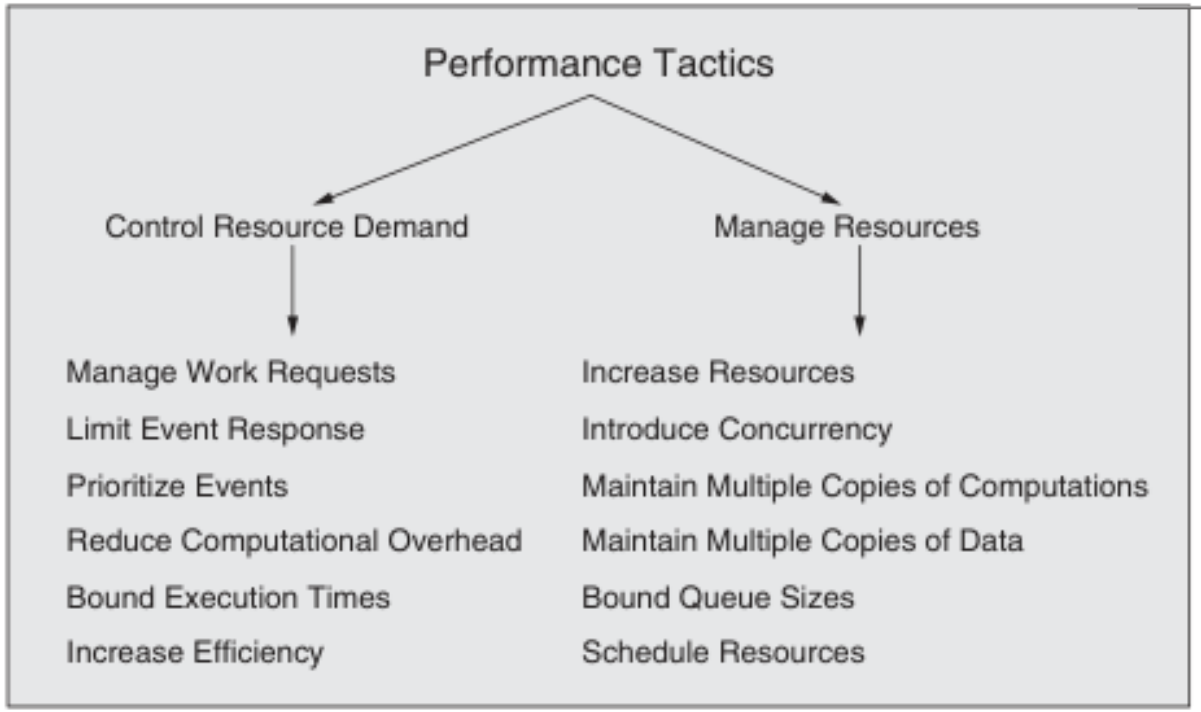


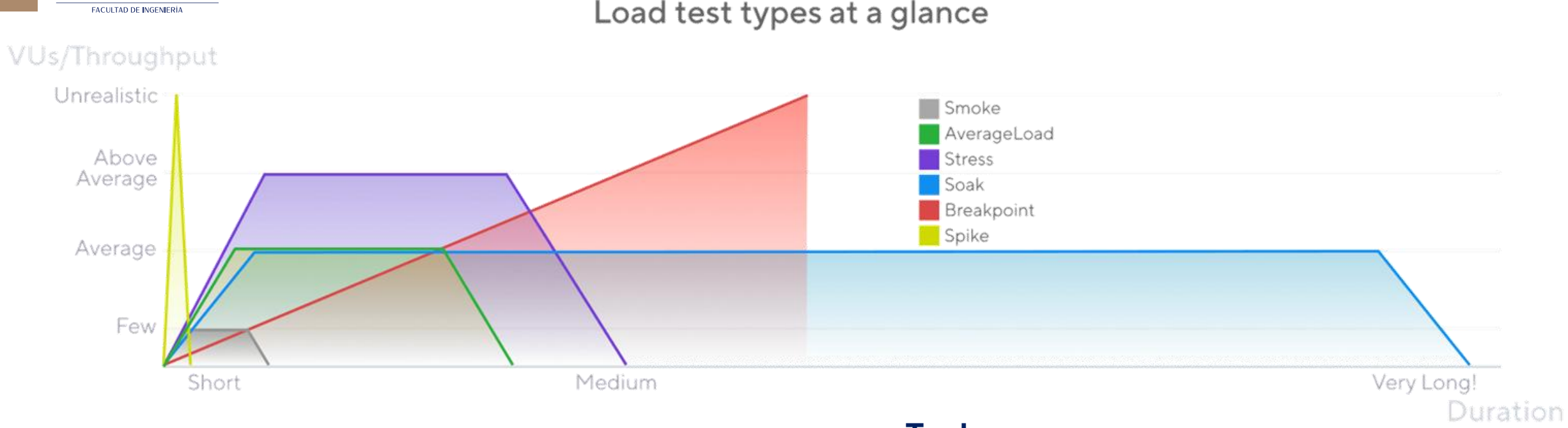
FIGURE 9.3 Performance tactics

Patterns

- Service Mesh (ms)
- Load Balancer
- Throttling
- Map Reduce

Universidad de La Sabana
FACULTAD DE INGENIERIA

Performance Measurements



Type	VUs/Throughput	Duration	When?
Smoke	Low	Short (seconds or minutes)	When the relevant system or application code changes. It checks functional logic, baseline metrics, and deviations
Average-load	Average production	Mid (5-60 minutes)	Often to check system maintains performance with average use
Stress	High (above average)	Mid (5-60 minutes)	When system may receive above-average loads to check how it manages
Soak	Average	Long (hours)	After changes to check system under prolonged continuous use
Spike	Very high	Short (a few minutes)	When the system prepares for seasonal events or receives frequent traffic peaks
Breakpoint	Increases until break	As long as necessary	A few times to find the upper limits of the system

Tools

APACHE JMeter™

Gatling LOAD TESTING

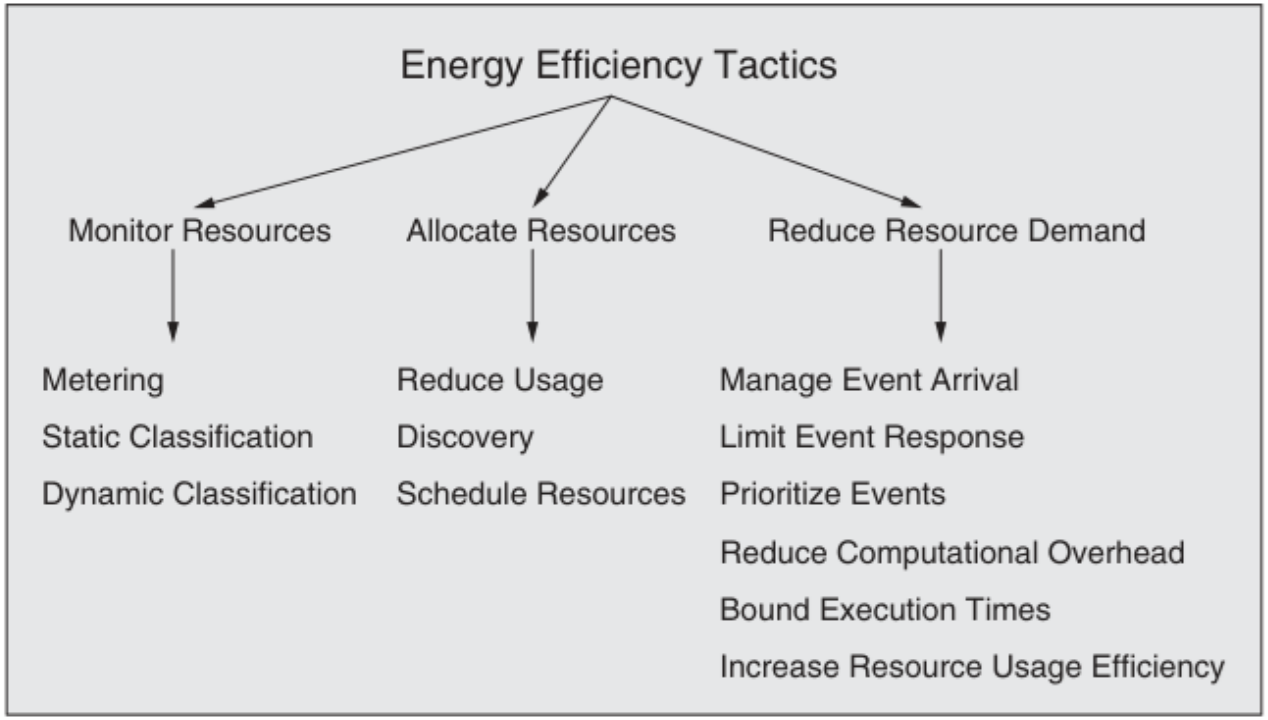
SoapUI

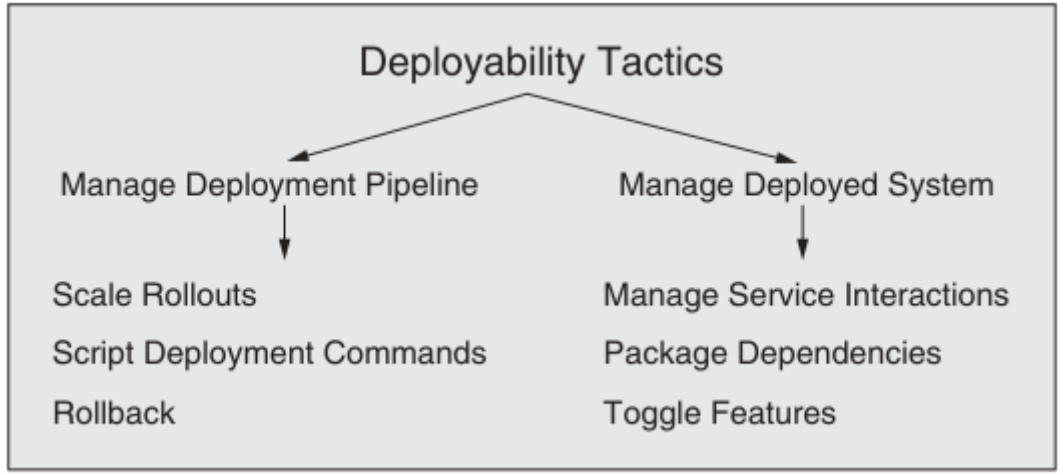
Azure Load Testing

Links

Universidad de La Sabana
FACULTAD DE INGENIERIA

Important QA I





FinOps interest:
- Maximize cost
- Cost avoidance

Really ?

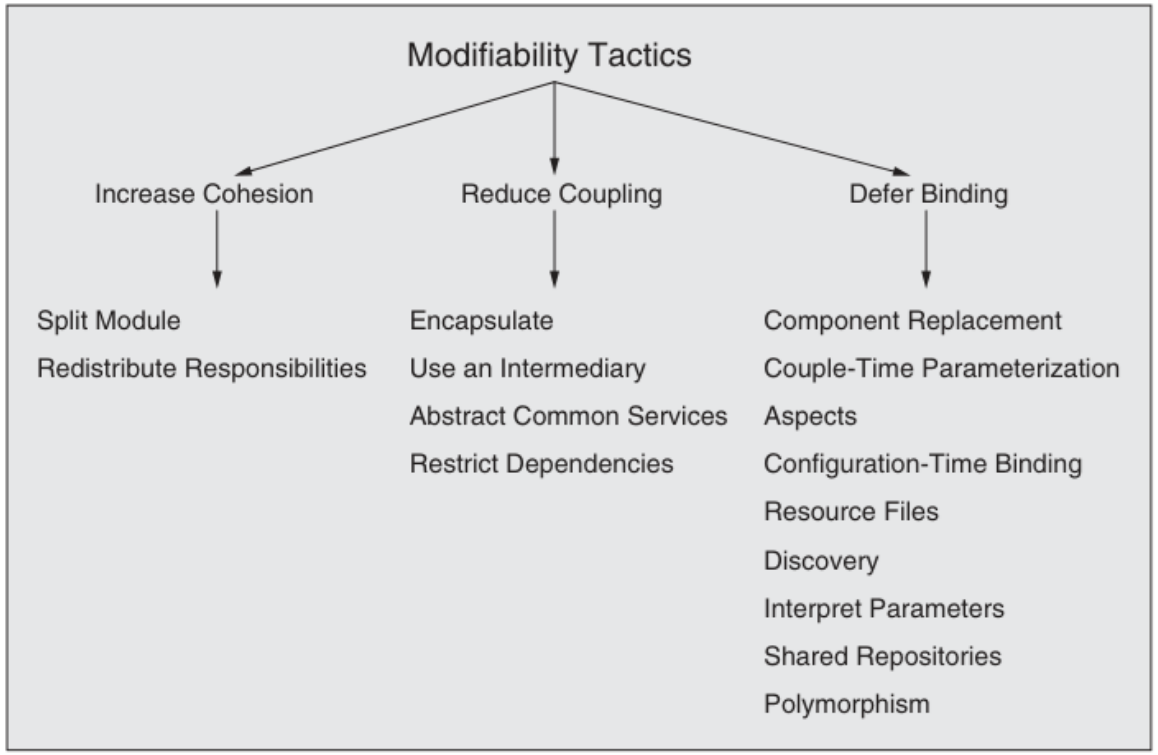
Same contention as Performance (*Control Resource Demand*)

Good DevOps Practices:
- Avoid downtime, frequent small changes
- Toggle Features (Low-Code)

Patterns:
- Microservices
- Complete Replacement: Blue/Green Deployment and Rolling Update
- Partial Replacement: Canary Testing and A/B Testing

Universidad de La Sabana
FACULTAD DE INGENIERIA

Important QA II



For N similar modifications, a simplified justification for a change mechanism is that

N * Cost of making change without the mechanism ≤
Cost of creating the mechanism + (N * cost of making the change using the mechanism)

Measurements

Response measure	The resources that were expended to make the change.	Cost in terms of: ▪ Number, size, complexity of affected artifacts ▪ Effort ▪ Elapsed time ▪ Money (direct outlay or opportunity cost) ▪ Extent to which this modification affects other functions or quality attributes ▪ New defects introduced ▪ How long it took the system to adapt

Patterns:

- Client-Server
- Microkernel (Plugin)
- Layers
- Publish-Subscribe

Measuring Software Maintainability



The following tactics can be used to bind values at compile time or build time:

- Component replacement (for example, in a build script or makefile)
- Compile-time parameterization
- Aspects

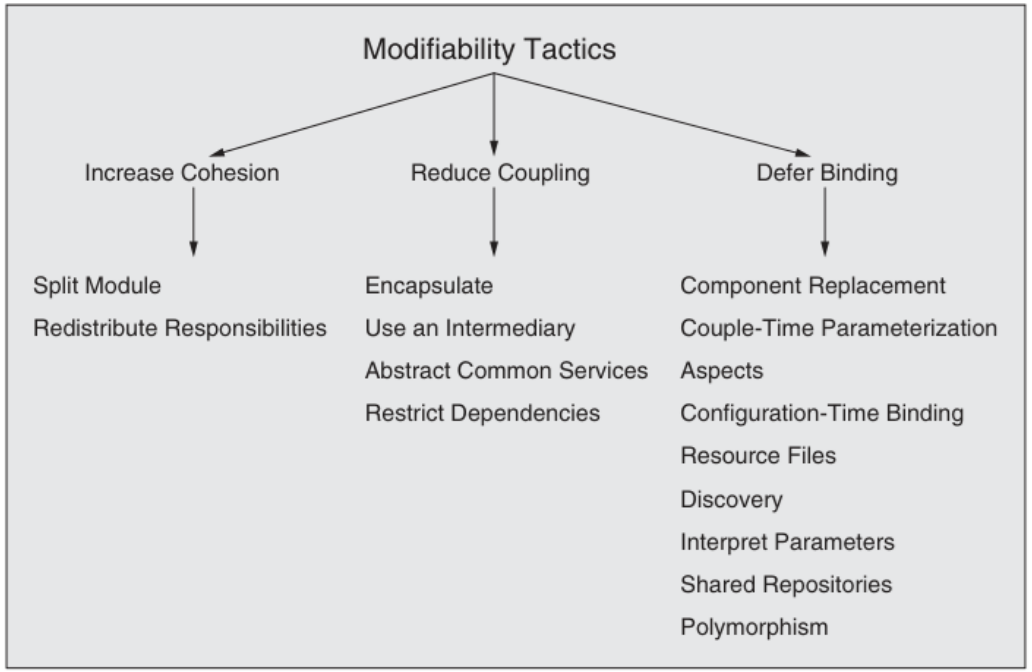
The following tactics are available to bind values at deployment, startup time, or initialization time:

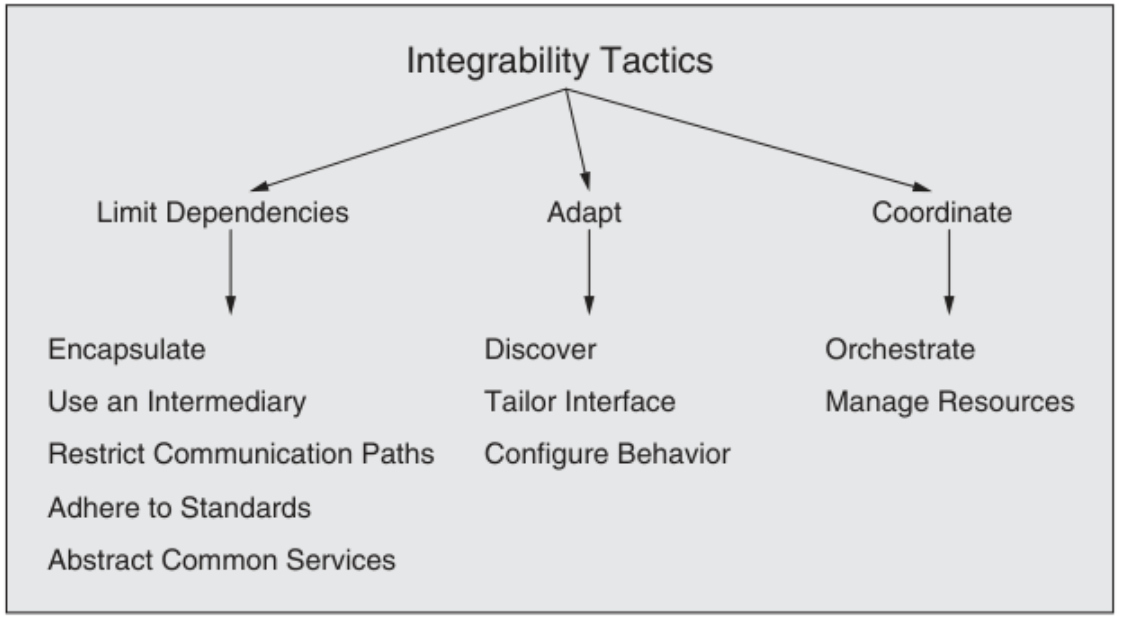
- Configuration-time binding
- Resource files

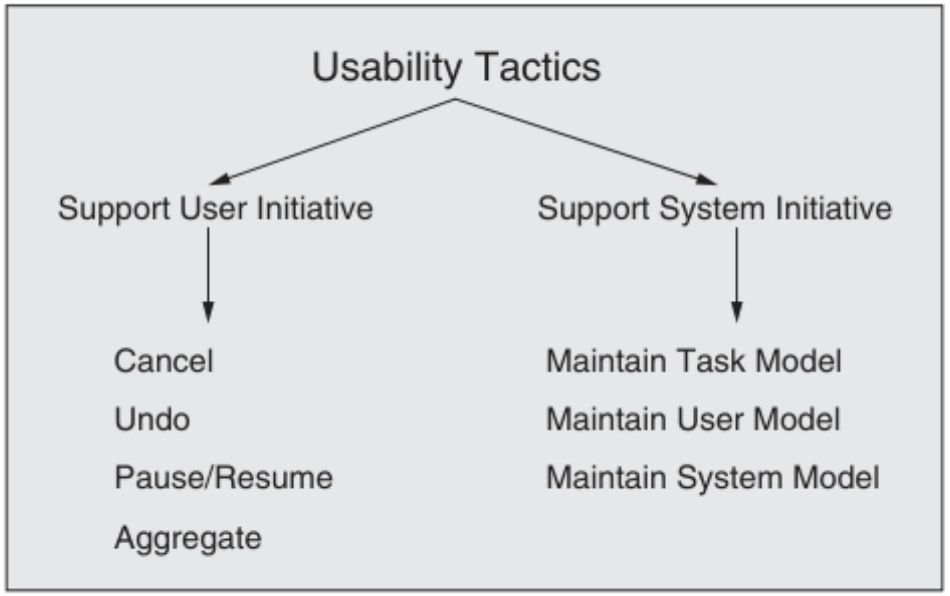
Tactics to bind values at runtime include the following:

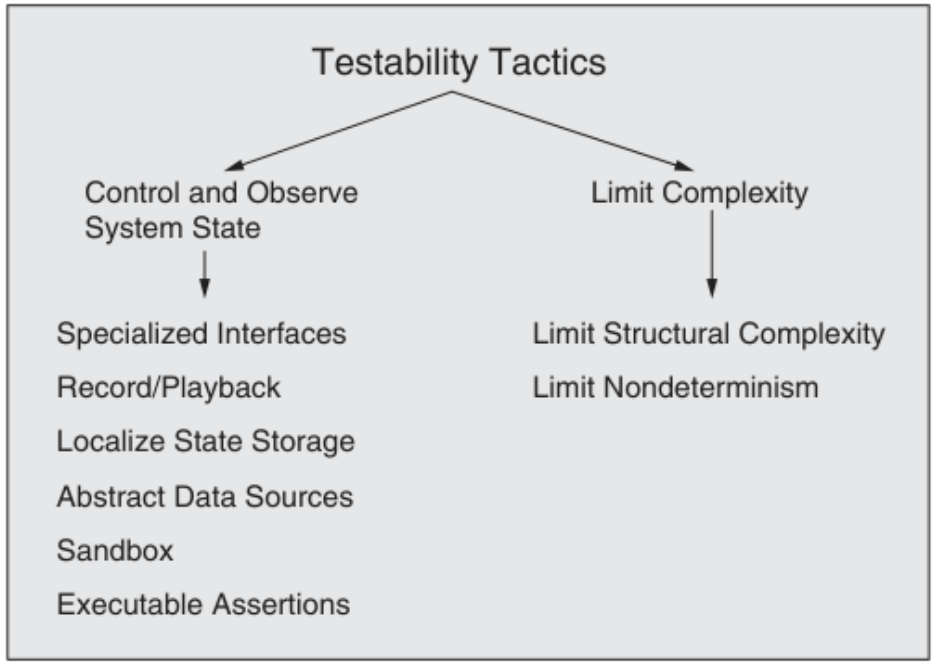
- Discovery (see Chapter 7)
- Interpret parameters
- Shared repositories
- Polymorphism

Other Tactics I

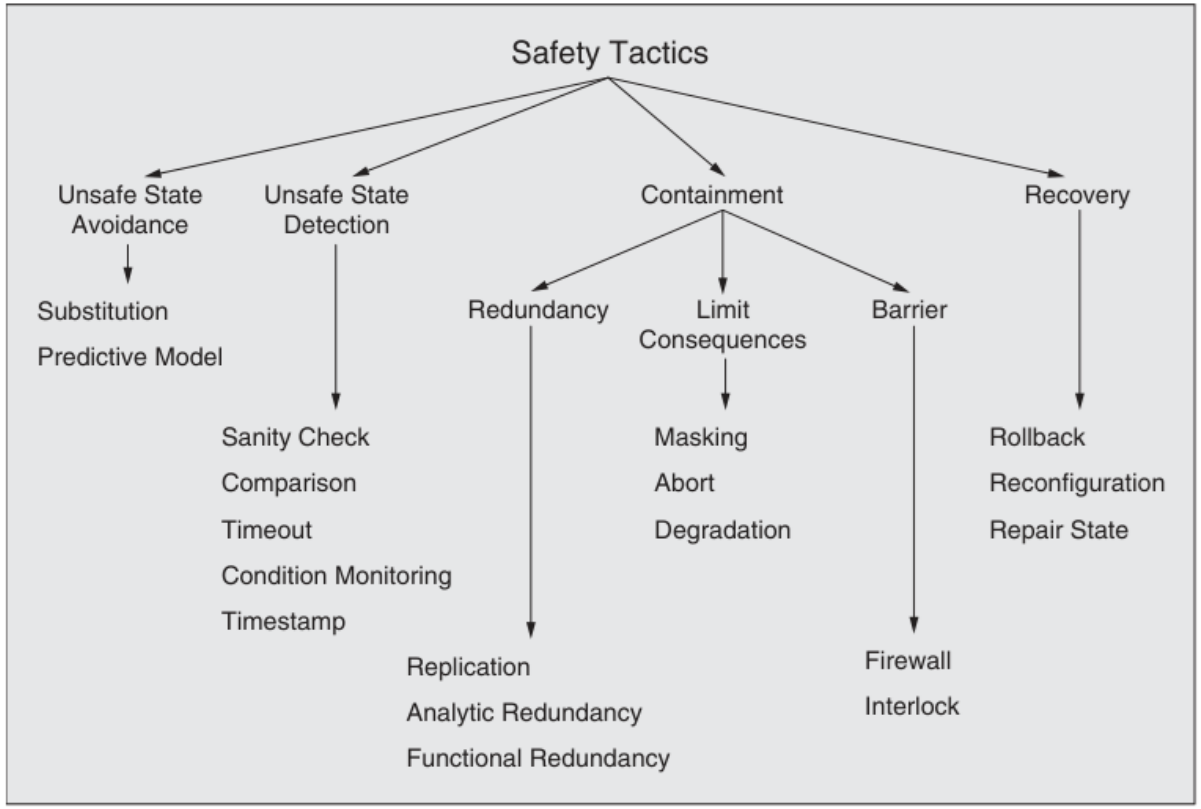


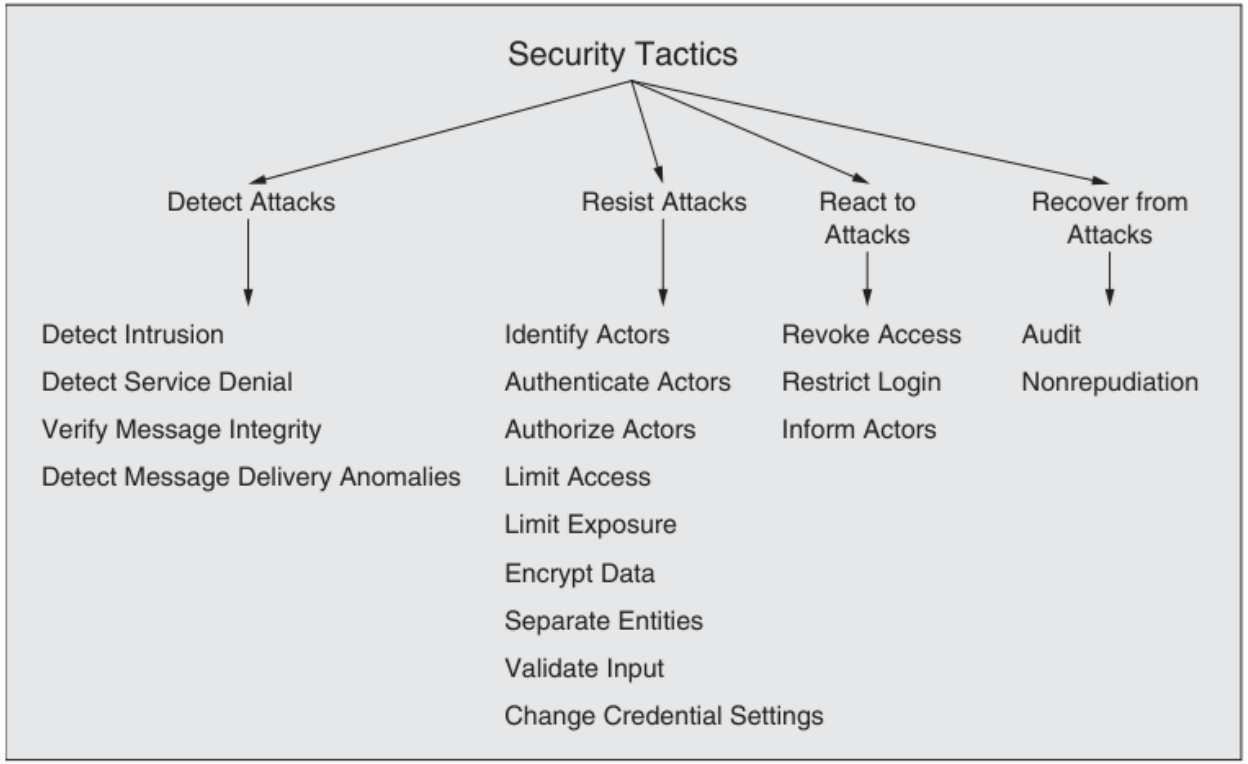






Other Tactics II





Tactic Questionnaires vs Well-Architected Tool

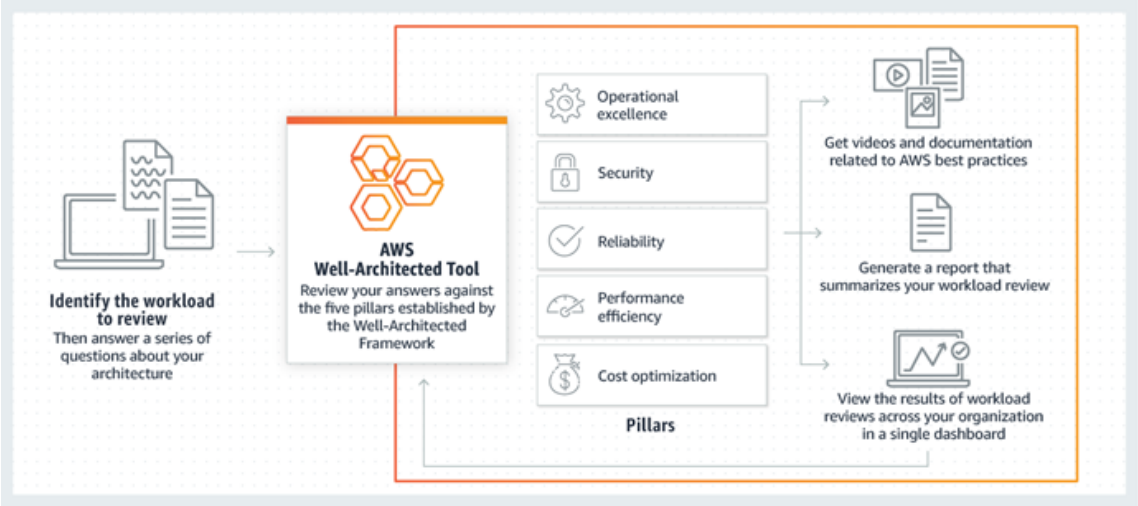
TABLE 5.2 Tactics-Based Questionnaire for Deployability

Tactics Groups	Tactics Question	Supported? (Y/N)	Risk	Design Decisions and Location	Rationale and Assumptions
Manage deployment pipeline	Do you **scale rollouts**, rolling out new releases gradually (in contrast to releasing in an all-or-nothing fashion)?				
	Are you able to automatically **roll back** deployed services if you determine that they are not operating in a satisfactory fashion?				
	Do you **script deployment commands** to automatically execute complex sequences of deployment instructions?				
Manage deployed system	Do you **manage service interactions** so that multiple versions of services can be safely deployed simultaneously?				
	Do you **package dependencies** so that services are deployed along with all of the libraries, OS versions, and utility containers that they depend on?				
	Do you employ **feature toggles** to automatically disable a newly released feature (rather than rolling back the newly deployed service) if the feature is determined to be problematic?				

Why?

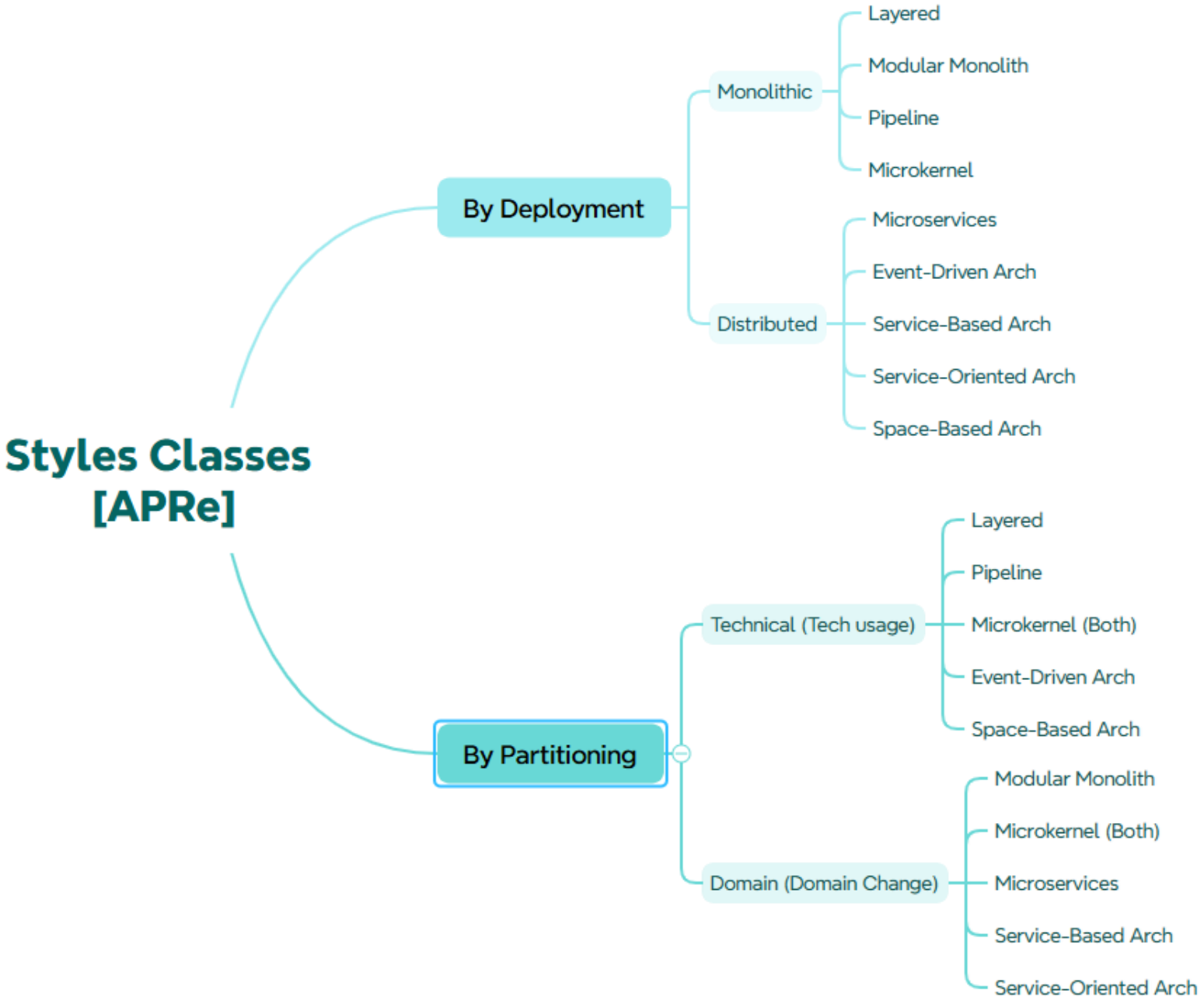
To implement: High/Medium/Low

Does current architecture support it ?



OMHO it can be do it before and after, to stablish a baseline or follow-up doc.

Classes of Arch Styles



It can be added by:
Database (Share, per service, event sourcing), Interaction (Req/Response, Event-Driven, Dataflow), etc..

Universidad de La Sabana
FACULTAD DE INGENIERIA

Layered Arch Style

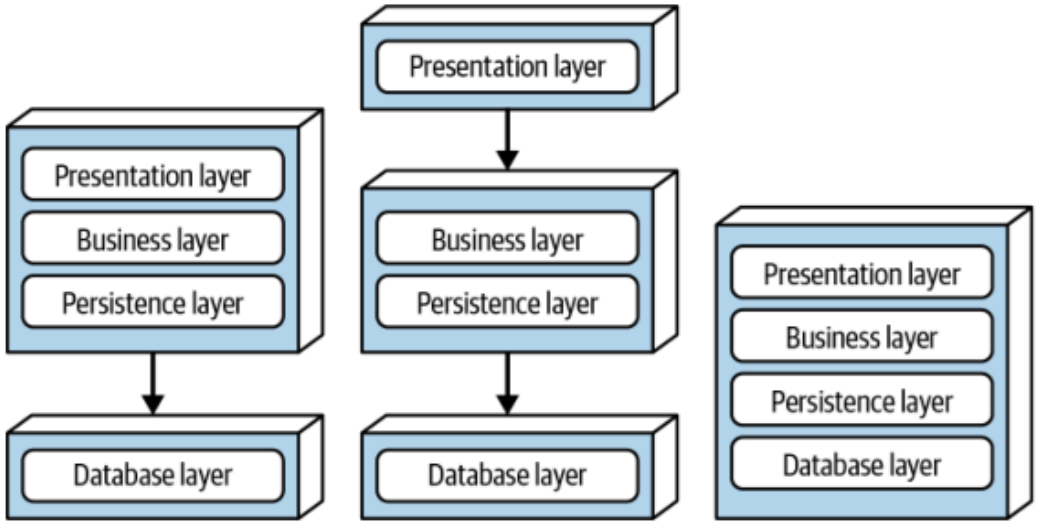


Figure 10-2. Physical topology (deployment) variants

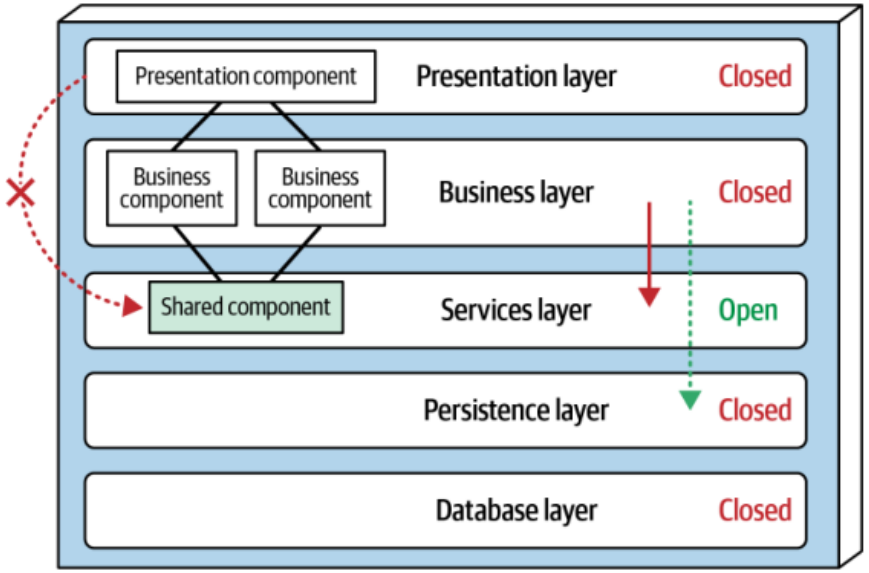


Figure 10-5. Adding a new Services layer to the architecture

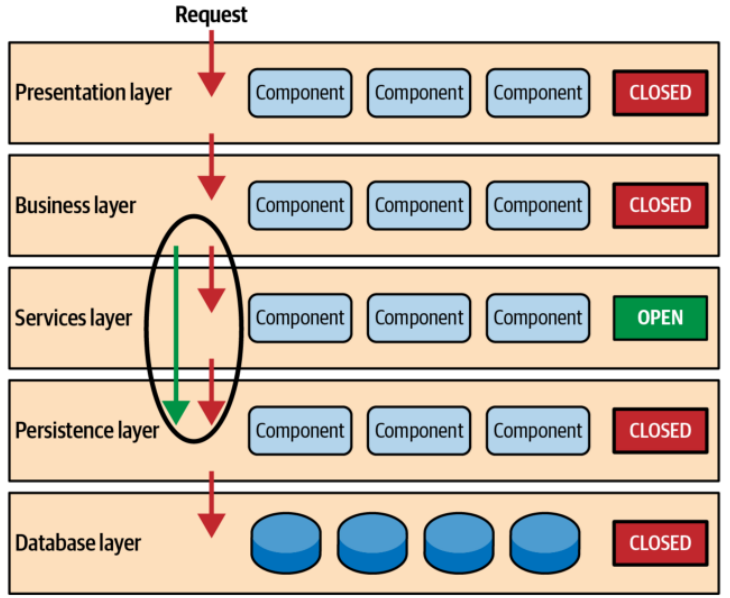


Figure 3-3. With open layers, the request can bypass the layer below it

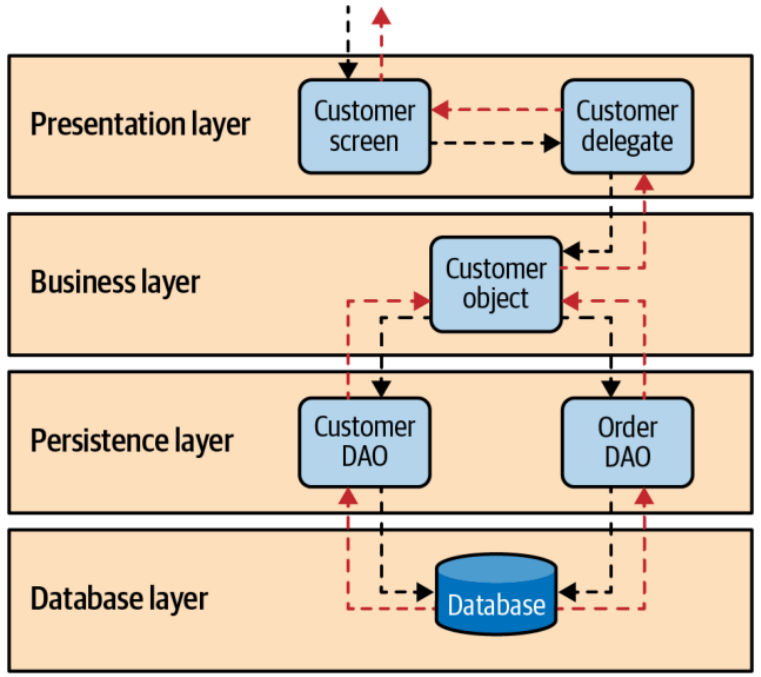
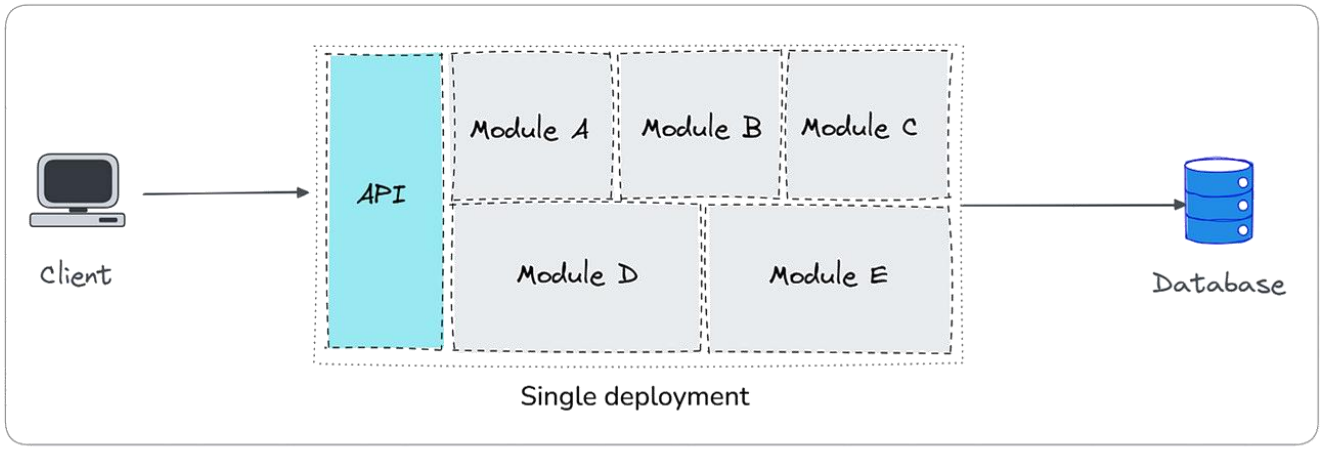


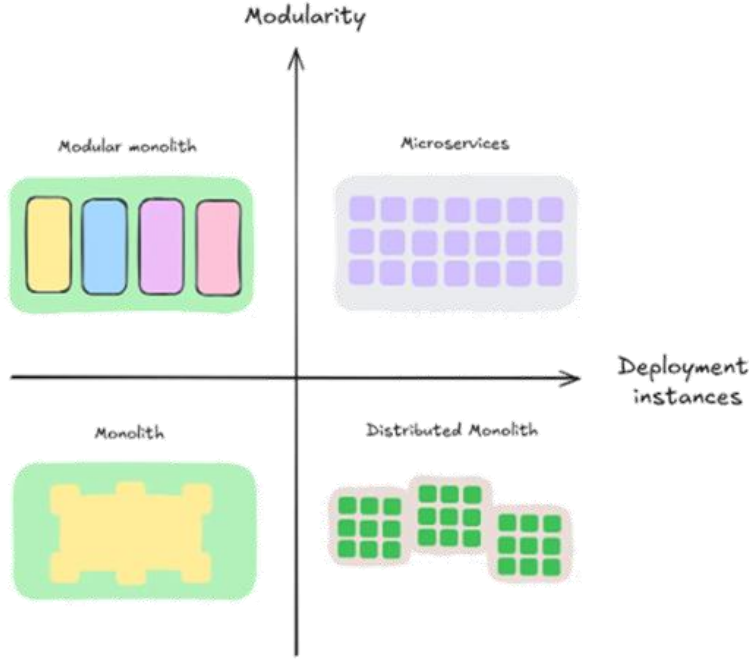
Figure 3-4. An example of the layered architecture

Universidad de La Sabana
FACULTAD DE INGENIERIA

Modular Monolith Arch Style

Modular Monolith





Modular Monolith Vs Microservices

Feature	Modular Monolith	Microservices
Deployment	Single unit	Multiple independent services
Complexity	Lower complexity	Higher due to distributed systems challenges
Performance	Faster (in-memory communication)	Slower (network calls between services)
Scalability	Limited to vertical scaling	Highly scalable with horizontal scaling
Operational Overhead	Lower (simpler infrastructure and monitoring)	Higher (orchestration, monitoring, service discovery needed)
Data Management	Single database	Distributed databases (eventual consistency issues)
Development Speed	Faster in early stages	Slower initially due to service separation and infrastructure setup
Flexibility	Can evolve into microservices if needed	Hard to transition back to a monolith
Fault Isolation	Limited, as all components share the same runtime	Better, as failures in one service do not affect others
Team Structure	Works well for small teams	Requires strong DevOps and service ownership culture
Communication	Internal function calls	Network-based APIs (REST, gRPC)
Maintainability	Easier to maintain in the early stages	More challenging without strong DevOps practices

Universidad de La Sabana
FACULTAD DE INGENIERIA

Event-Driven Arch (EDA) Style I

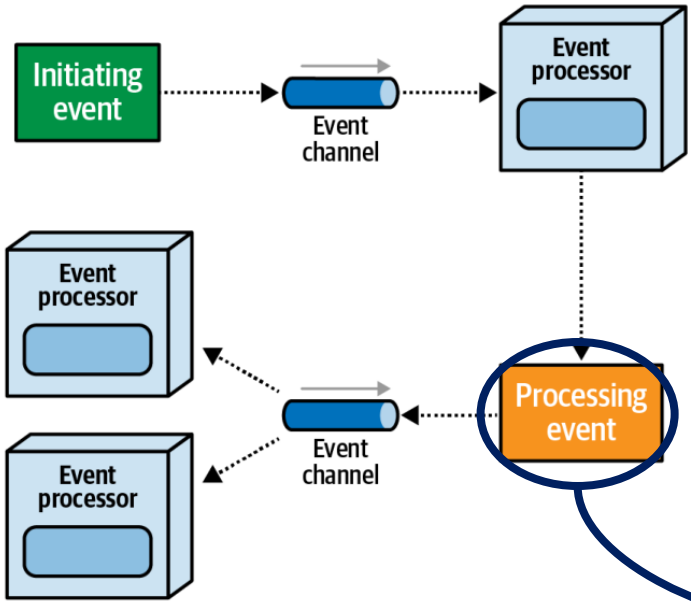


Figure 5-1. The main components of event-driven architecture

Same thing

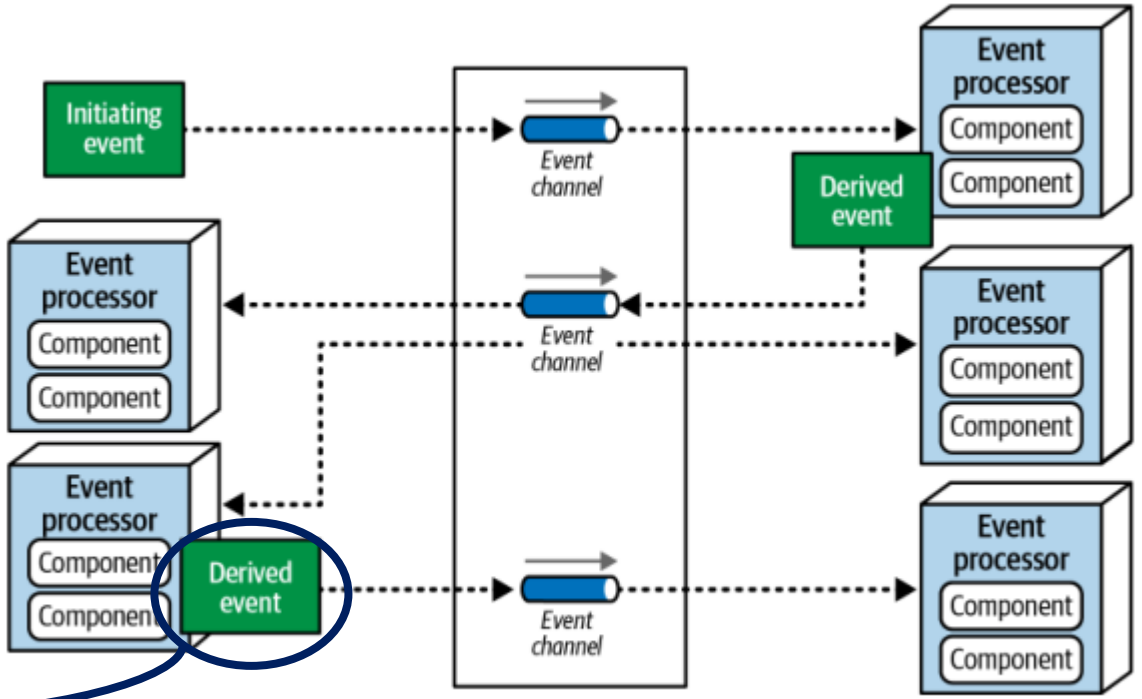


Figure 15-2. Basic topology of an event-driven architecture

Event-Driven Arch (EDA) Style II

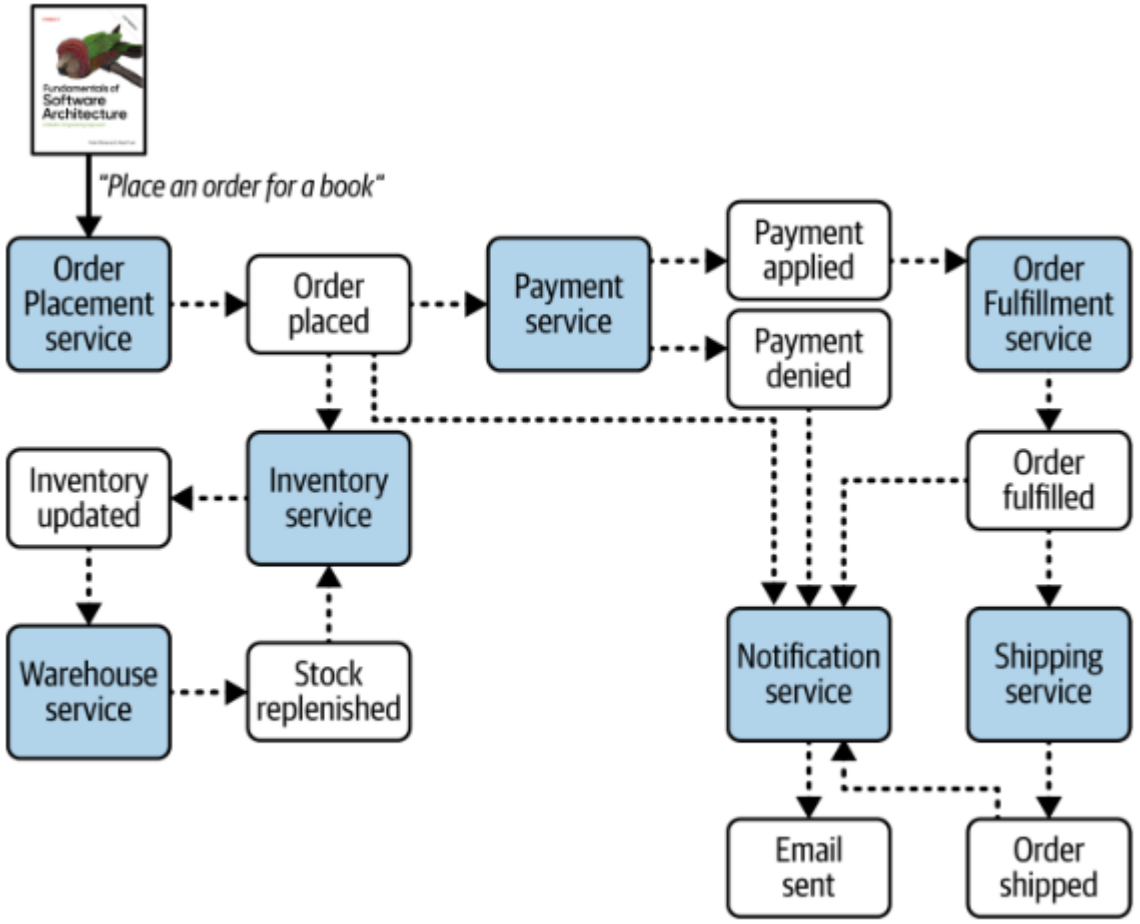


Figure 15-3. Example of the event-driven architecture topology

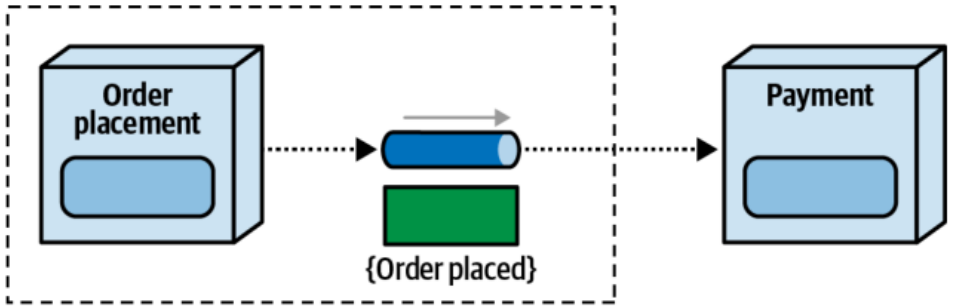


Figure 5-3. With events, the sender owns the event channel and contract

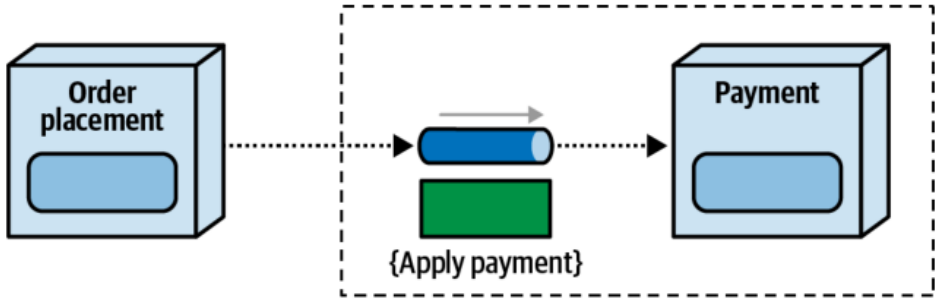


Figure 5-4. With messages, the receiver owns the message channel and contract

Universidad de La Sabana
FACULTAD DE INGENIERIA

Microservices, First Approach I

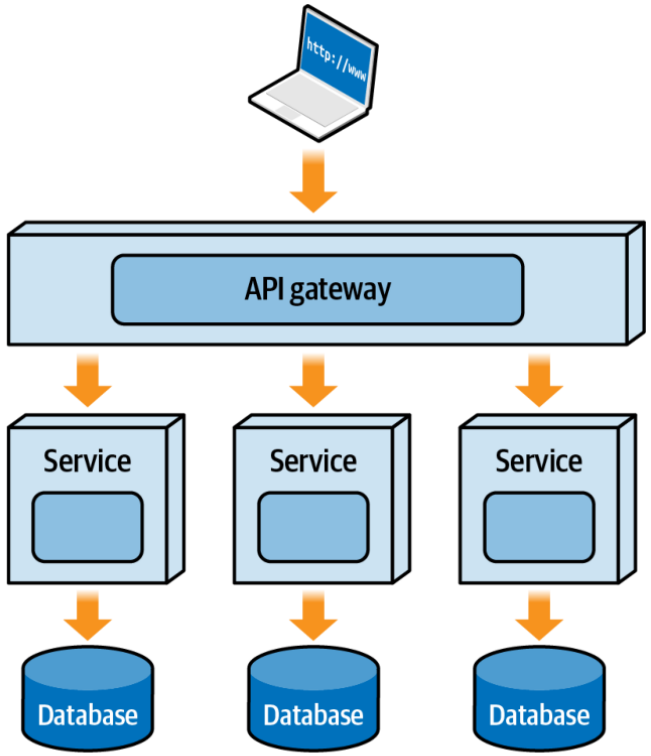


Figure 6-1. The basic topology of the microservices architecture style

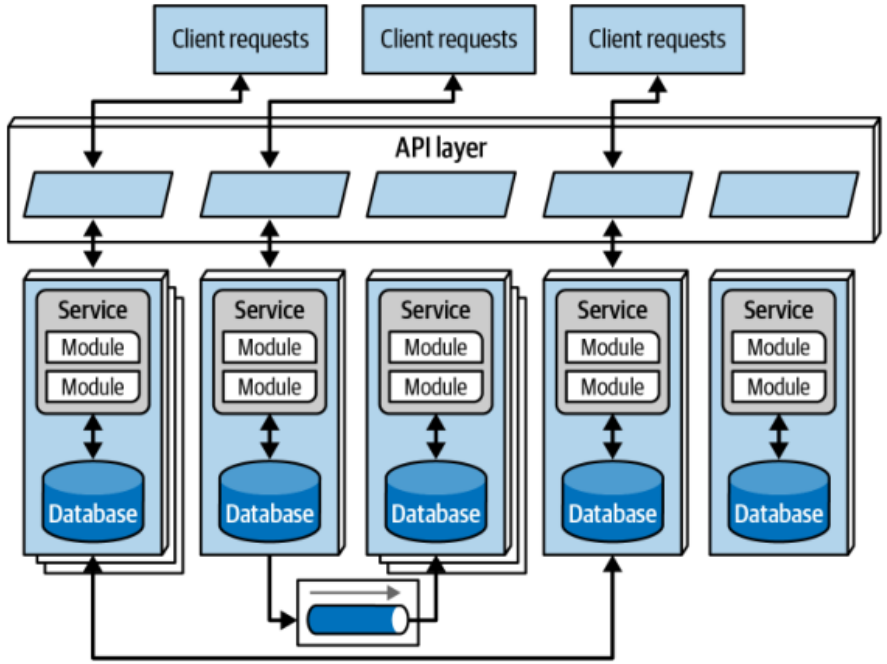


Figure 18-1. The topology of the microservices architecture style

Microservices, First Approach II

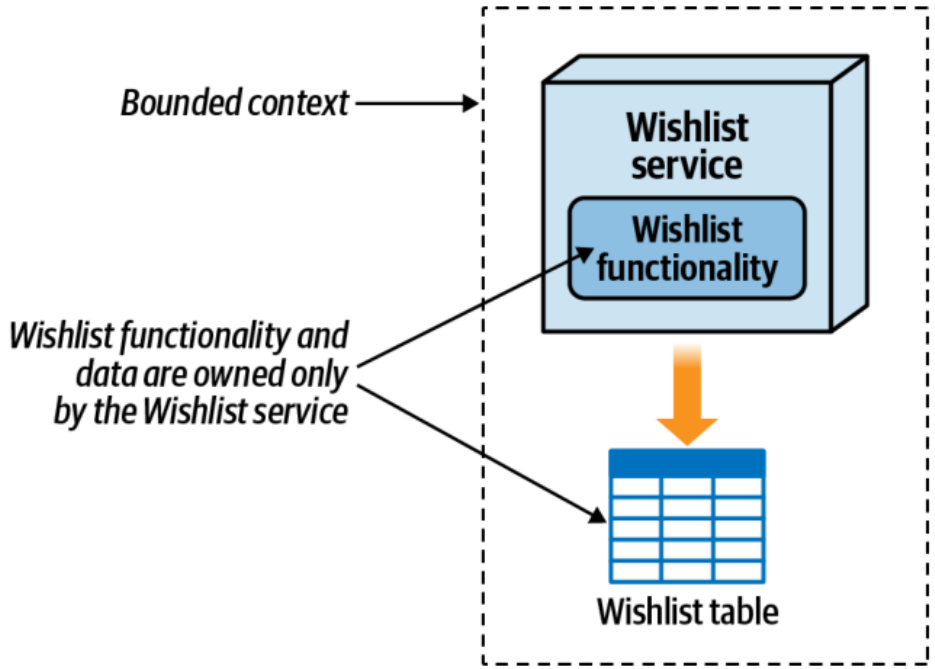


Figure 6-2. A bounded context includes the source code and corresponding data for a given domain or subdomain

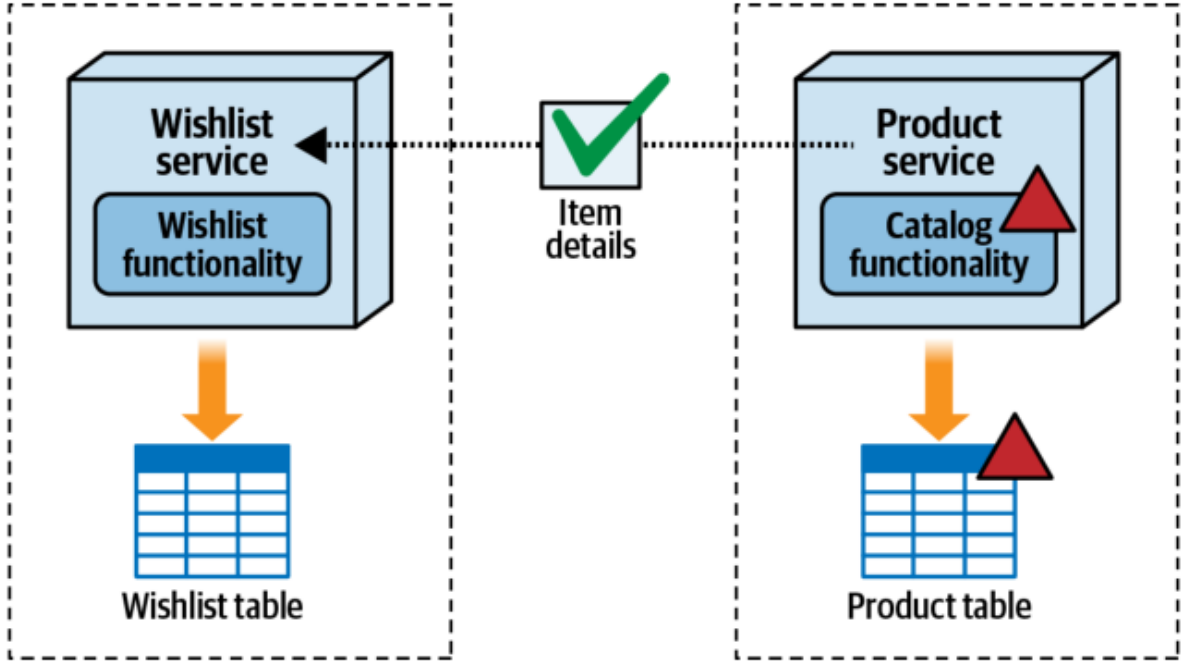


Figure 6-3. The bounded context usually isolates changes to just the service owning the data

Pipeline Arch Style

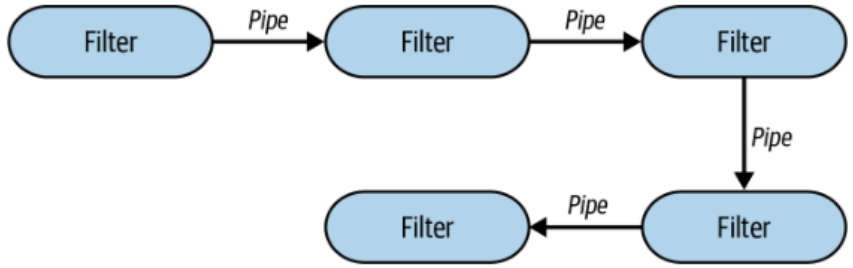


Figure 12-1. Basic topology for pipeline architecture

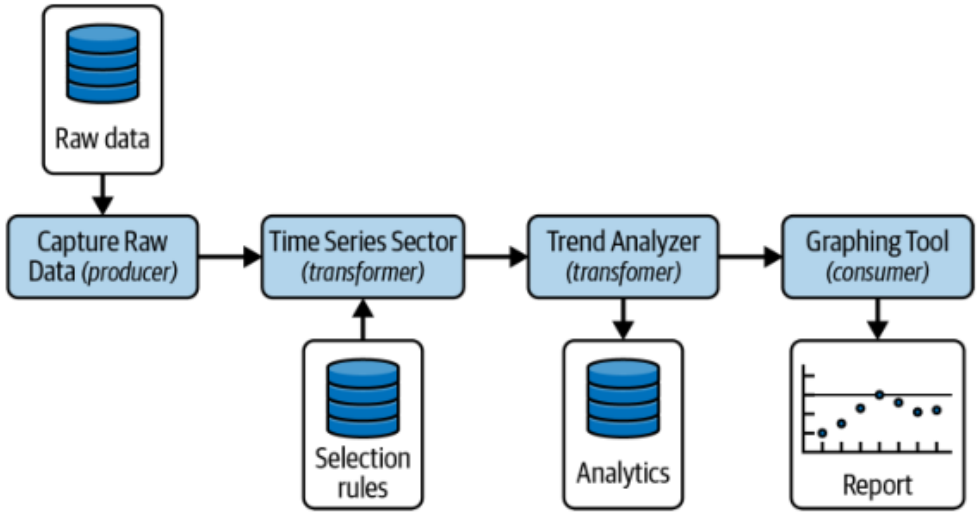
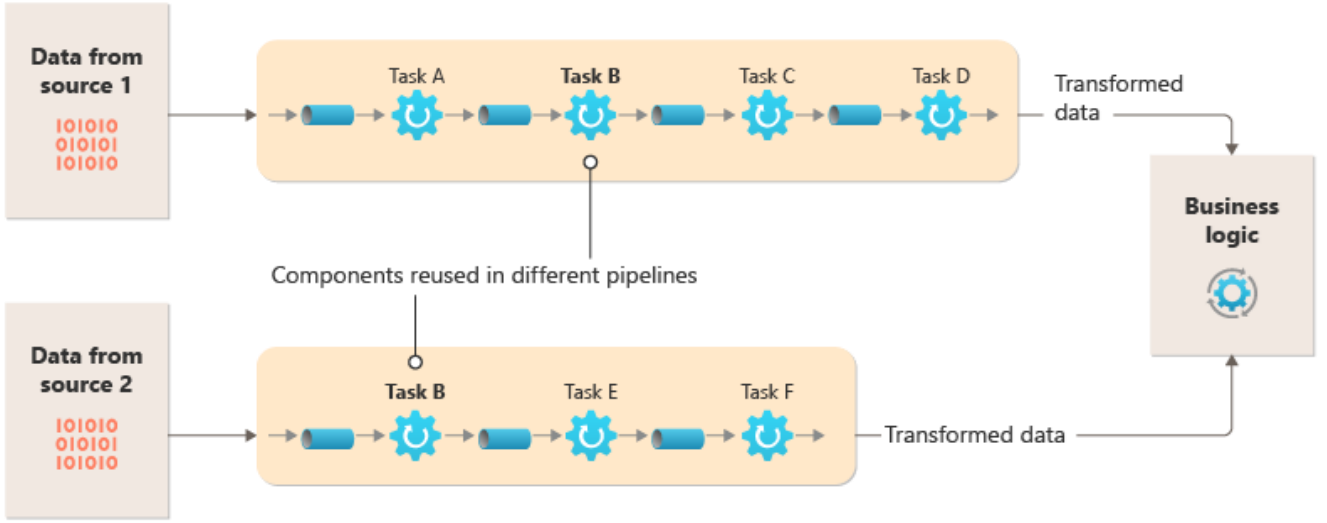
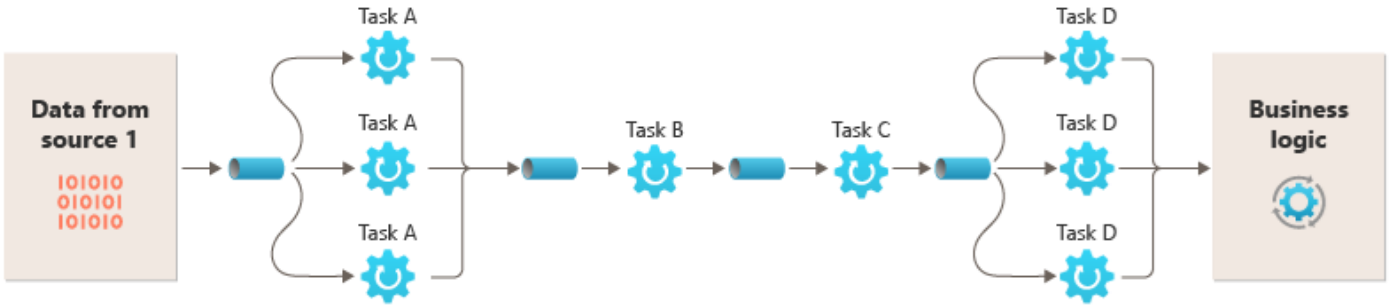


Figure 12-2. Pipeline architectures can have a single database or many





Universidad de La Sabana
FACULTAD DE INGENIERIA

Space-Based Arch Style

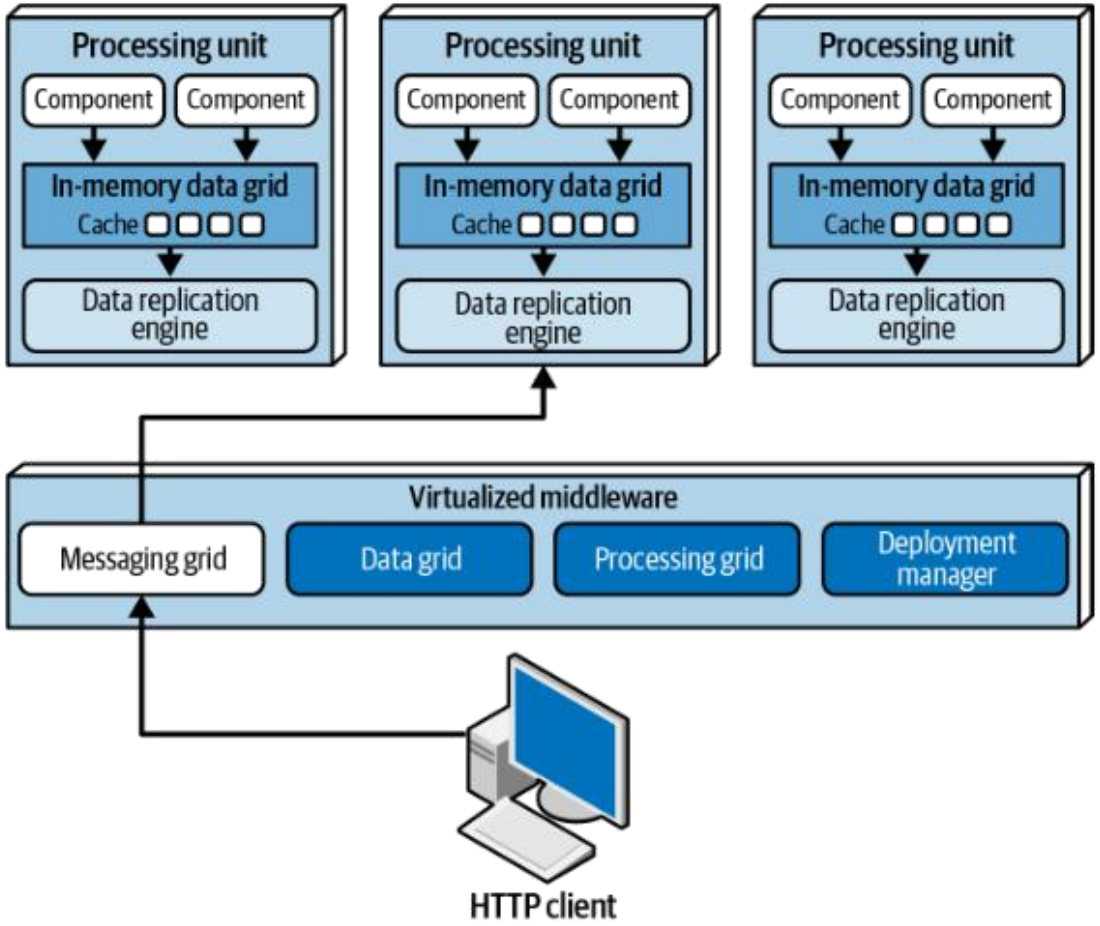


Figure 16-4. The messaging grid handles requests and session state

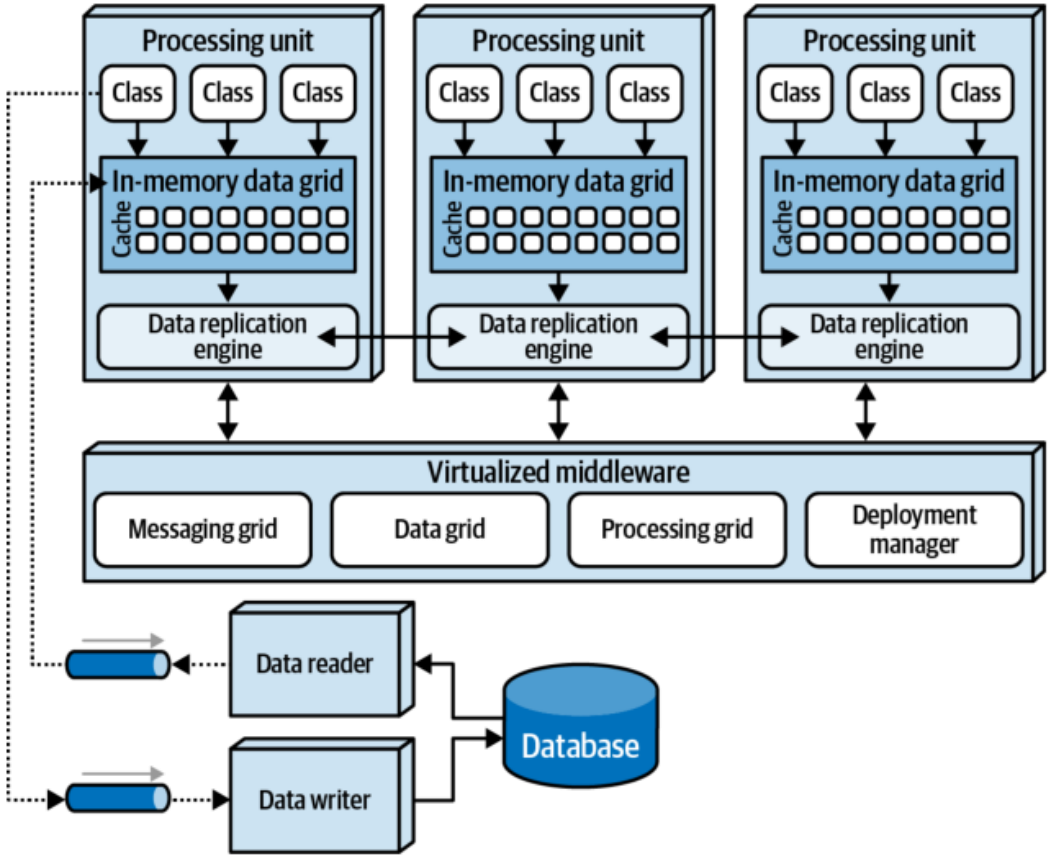


Figure 7-2. The space-based architecture style

[FoSA] and [APRe]

Orchestration-Driven SOA Style

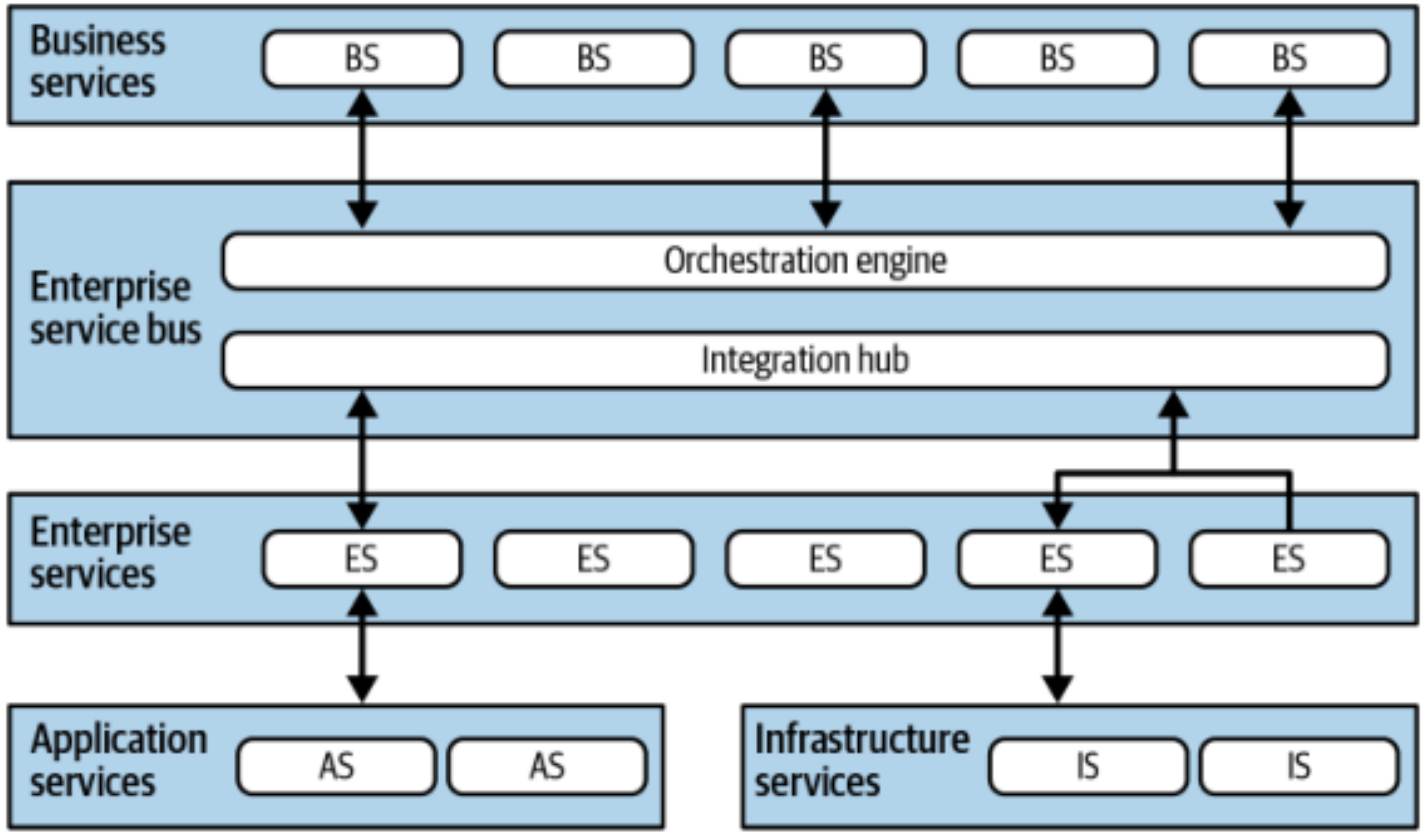
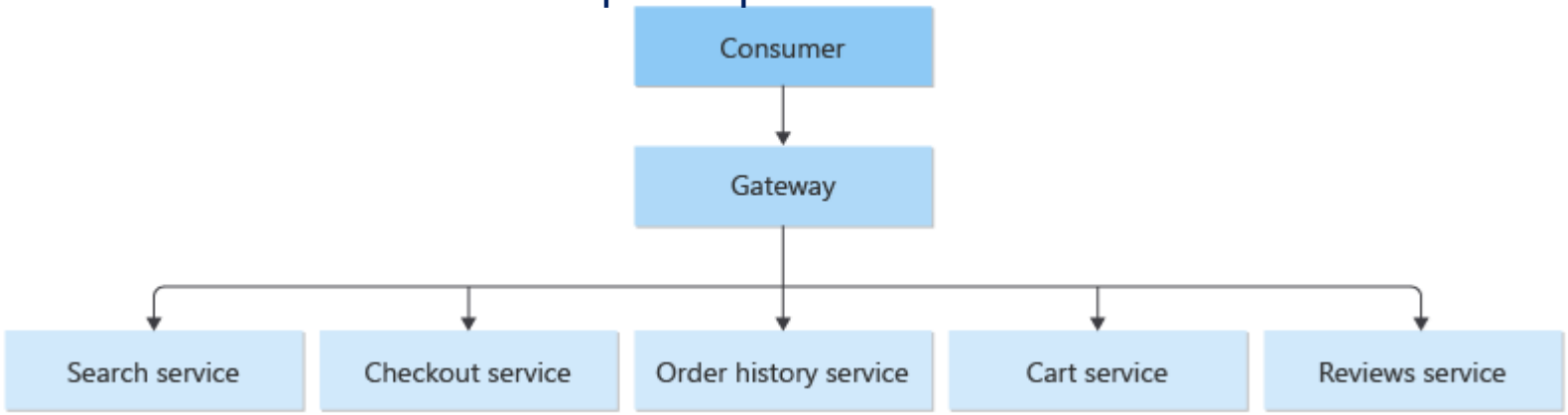


Figure 17-1. Topology of orchestration-driven service-oriented architecture

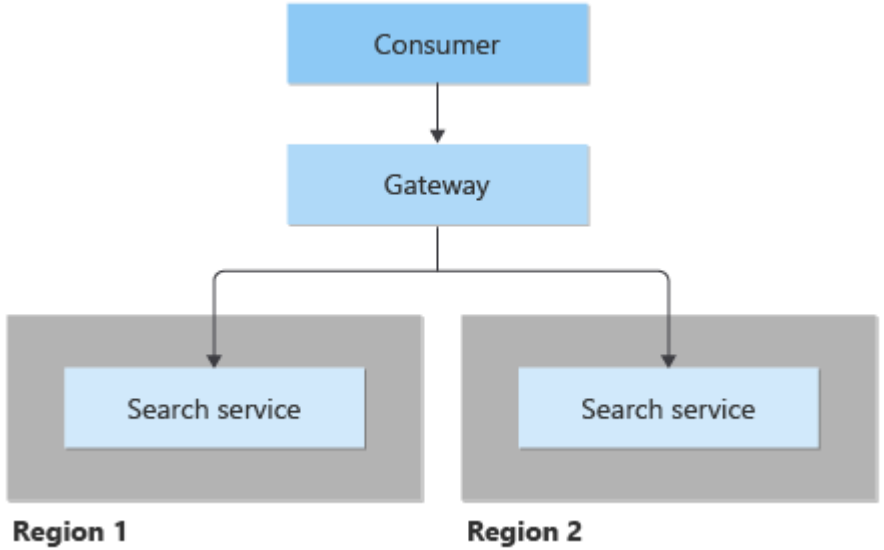
Universidad de La Sabana
FACULTAD DE INGENIERIA

Gateway Routing pattern

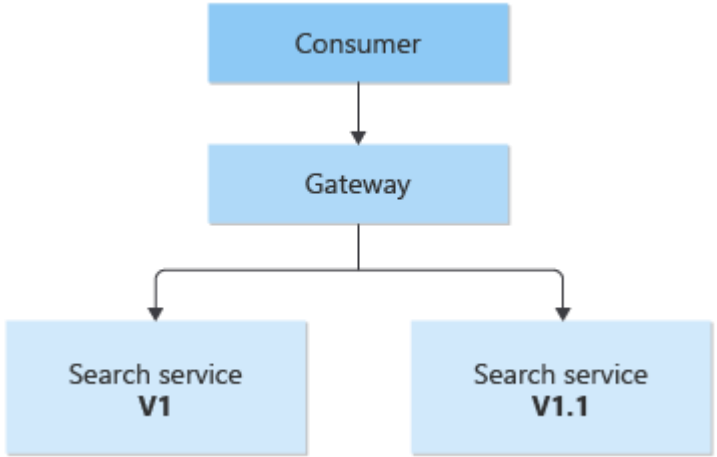
Multiple disparate services



Multiple instances of the same service



Multiple versions of the same service

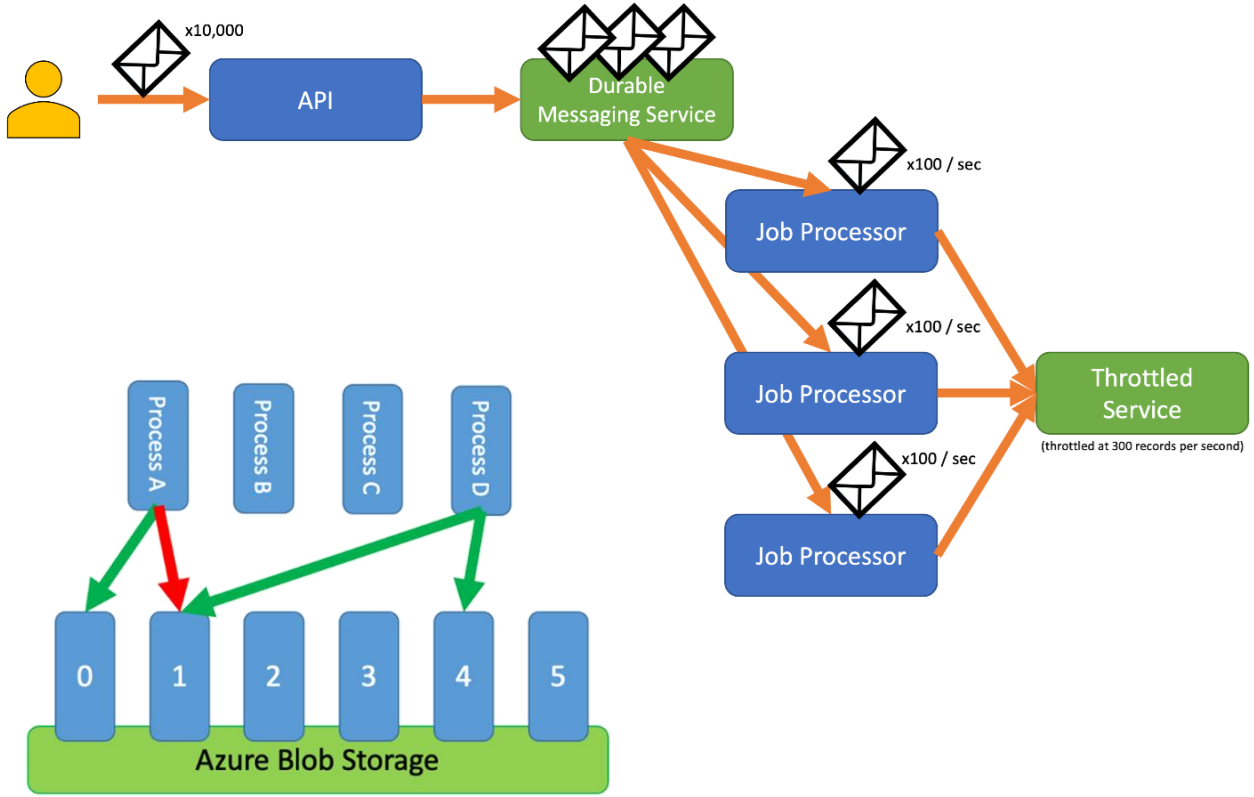


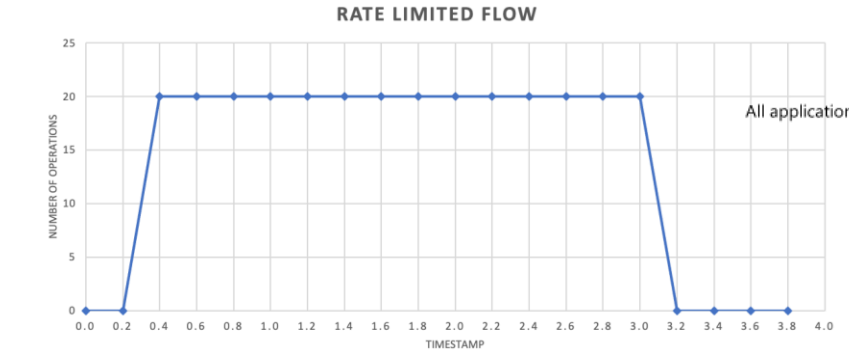
[AZDe]

Universidad de La Sabana
FACULTAD DE INGENIERIA

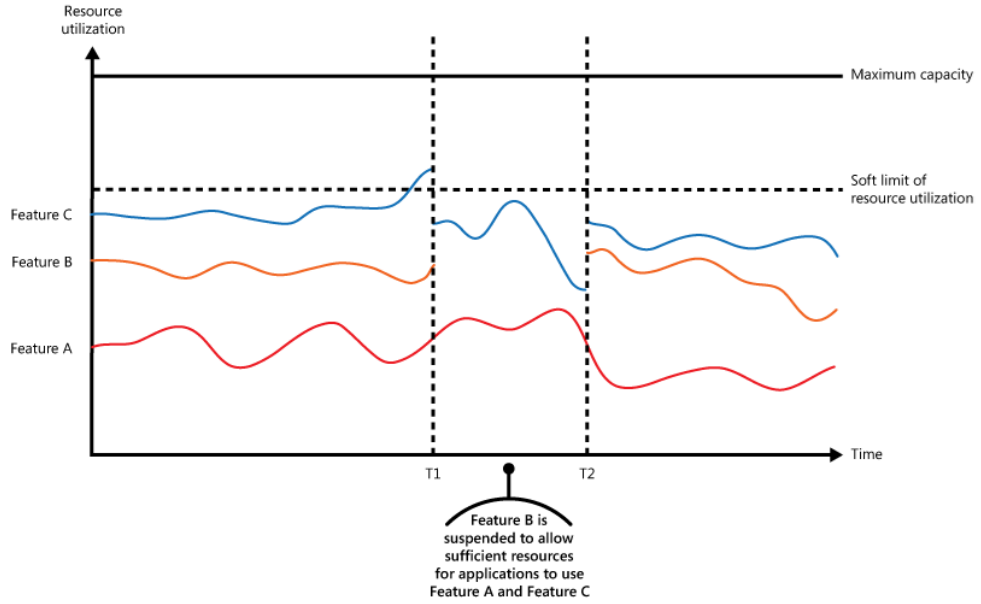
Rate Limiting/Throttling pattern

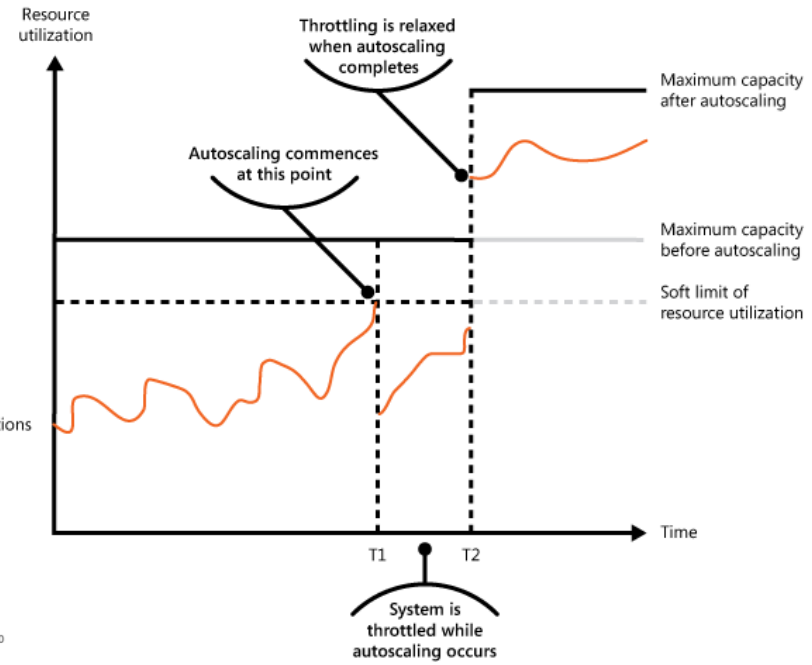
Rate Limiting Pattern





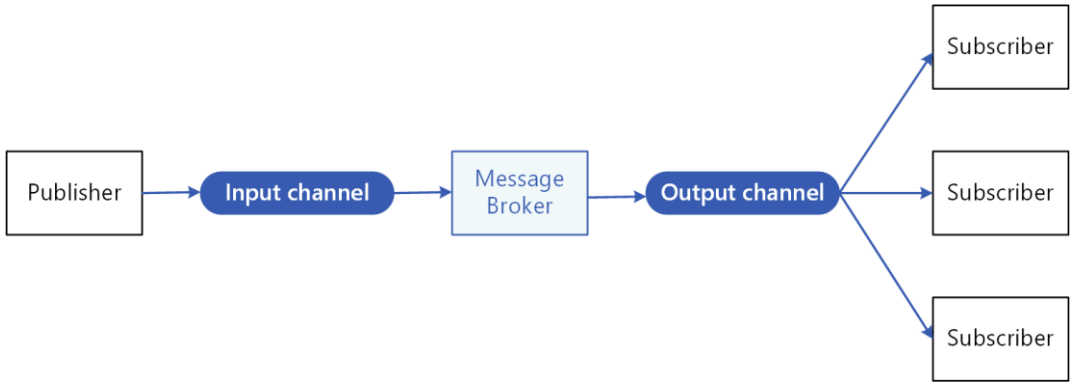
Throttling

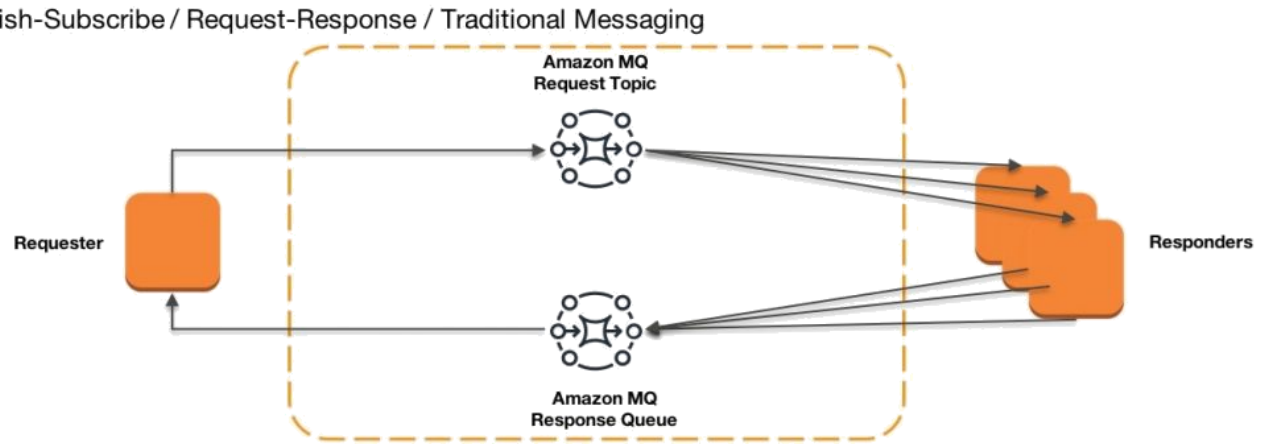


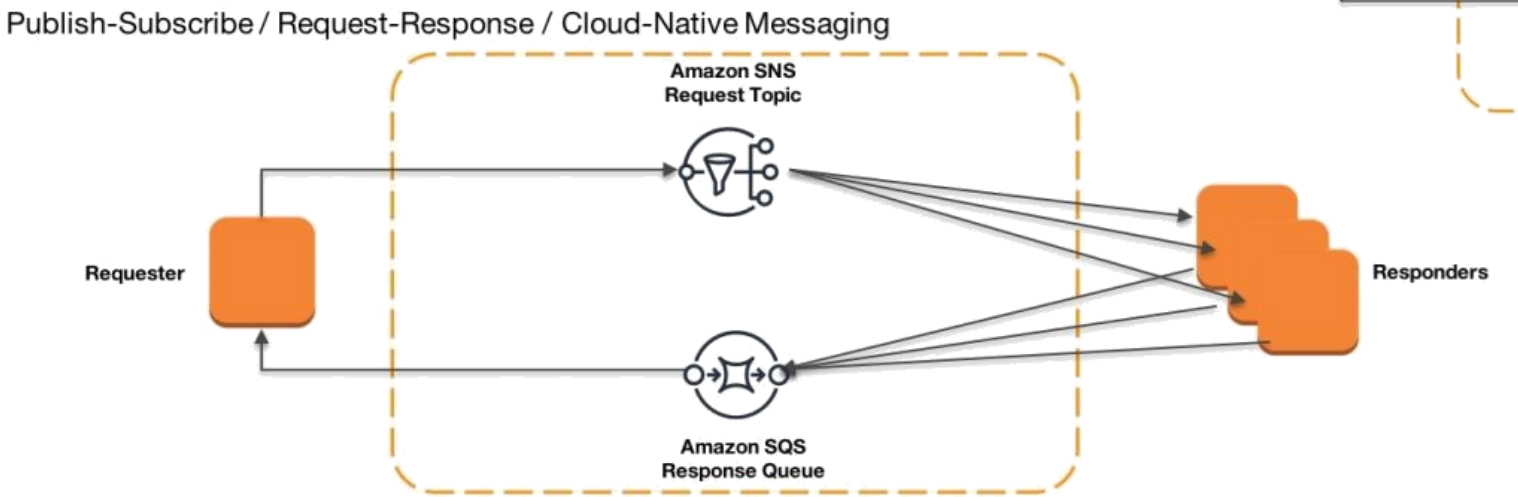


Universidad de La Sabana
FACULTAD DE INGENIERIA

Publisher-Subscriber pattern

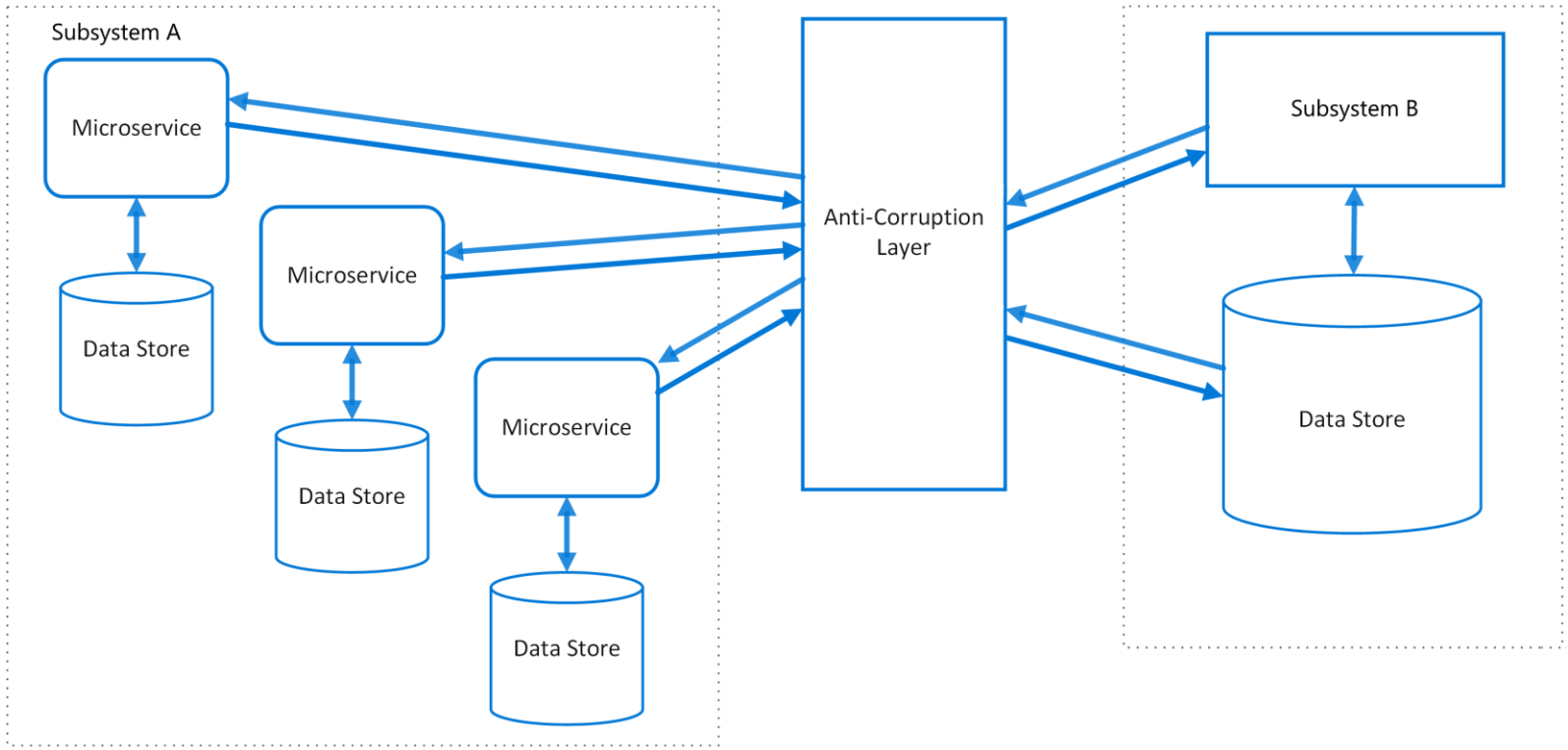






[AZDe] and link

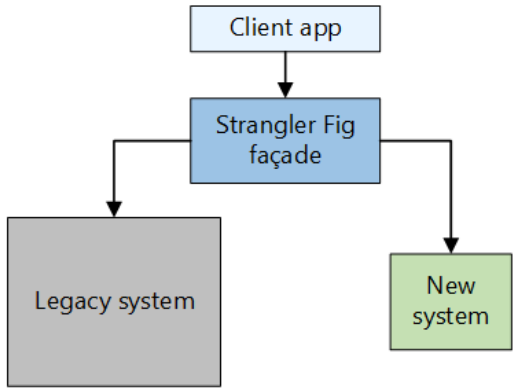
Anti-corruption Layer pattern



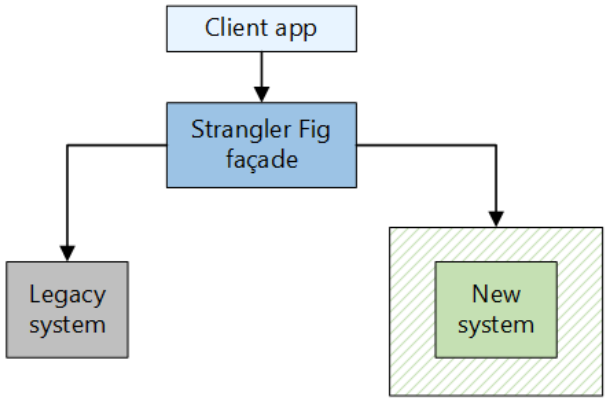
Universidad de La Sabana
FACULTAD DE INGENIERIA

Strangler Fig pattern

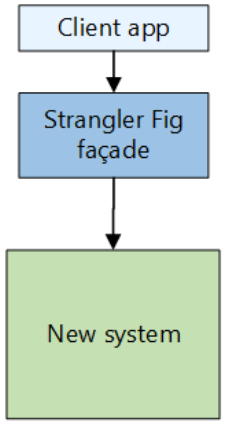
1 A façade routes client requests between the legacy system and the new system



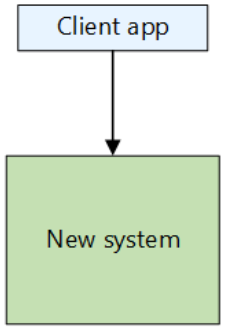
2 Incremental decomposition shifts functionality from the legacy system to the new system



3 The legacy system is fully decommissioned and has no dependencies



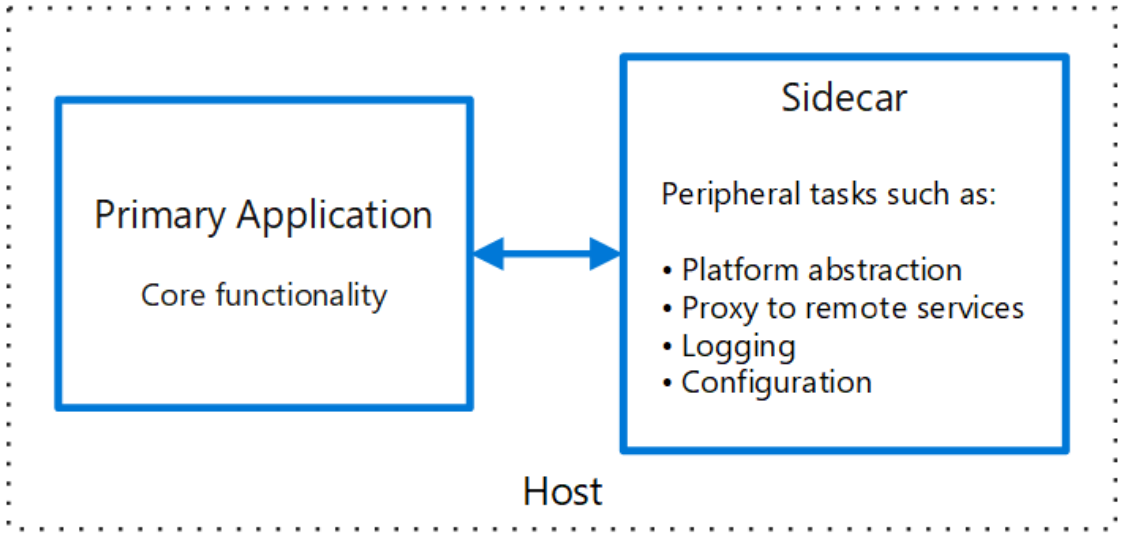
4 The façade is removed, and the client interacts directly with the new system



Microsoft Azure

[AZDe]

Sidecar



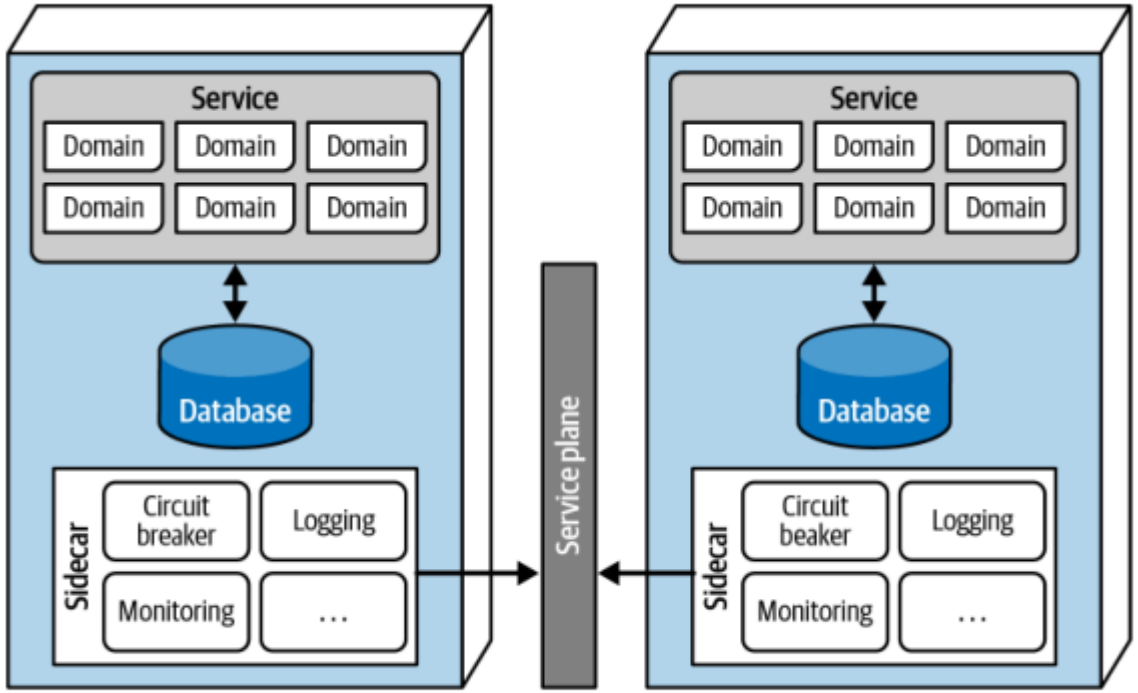


Figure 18-3. The service plane connects the sidecars in a service mesh

Service Mesh

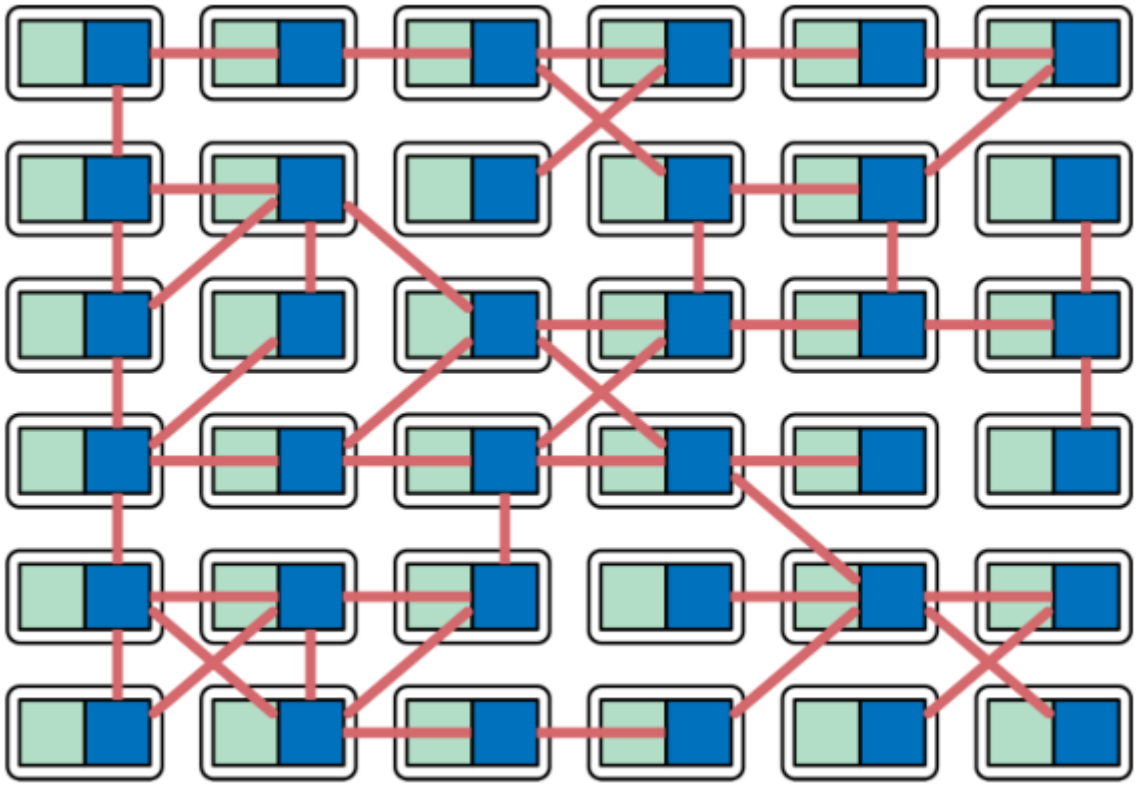
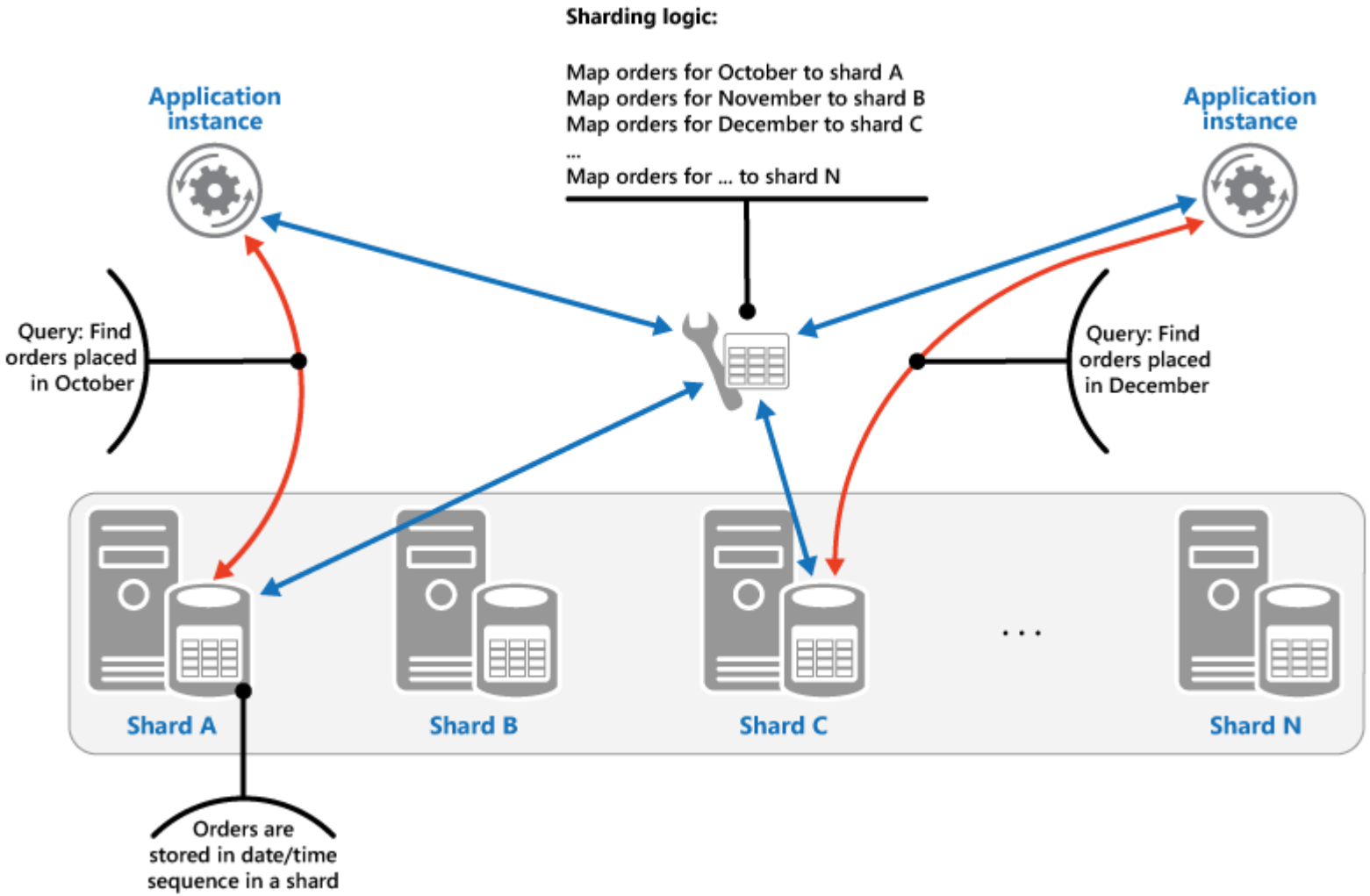


Figure 18-4. The service mesh forms a holistic view of the operational aspect of microservices

Universidad de La Sabana
FACULTAD DE INGENIERIA

Sharding pattern



Command Query Responsibility Segregation (CQRS)

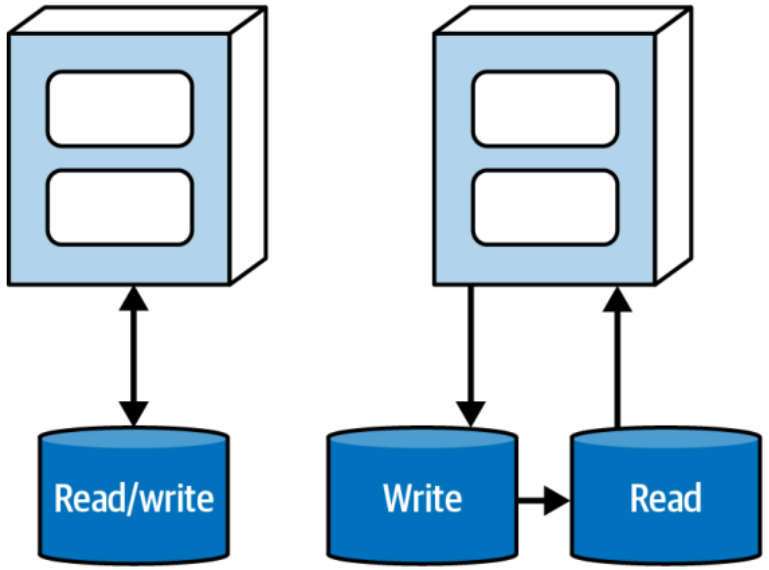
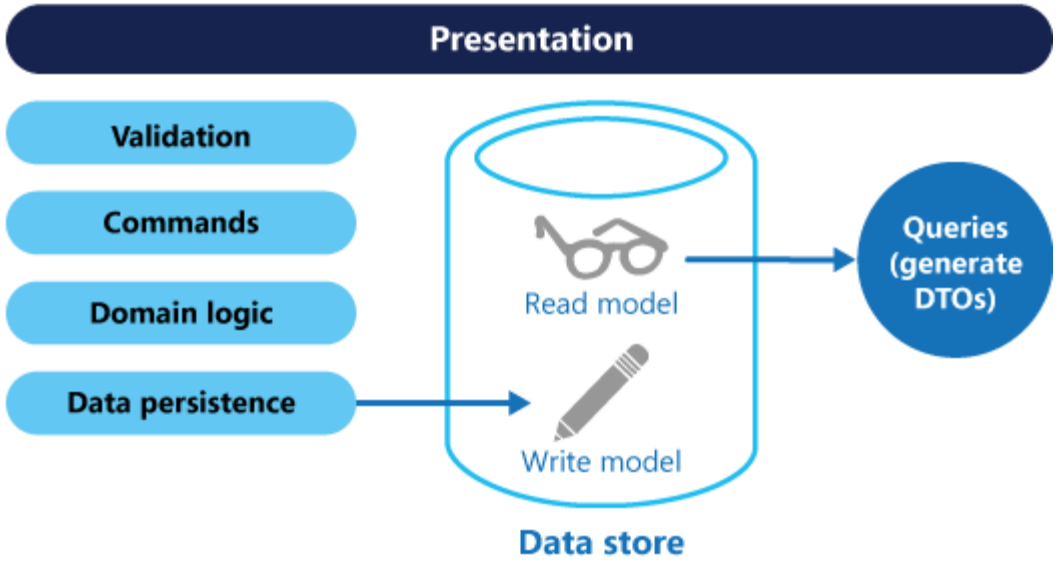
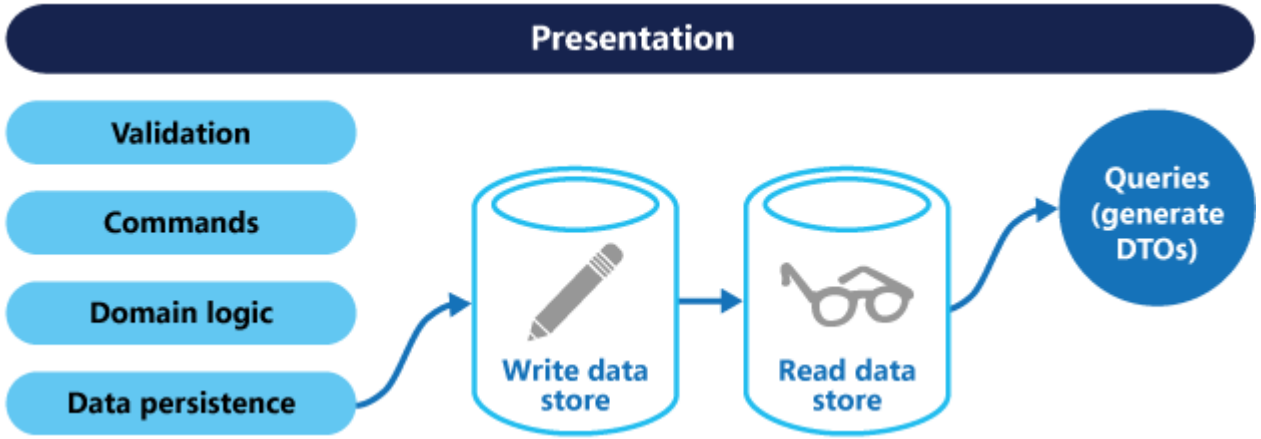


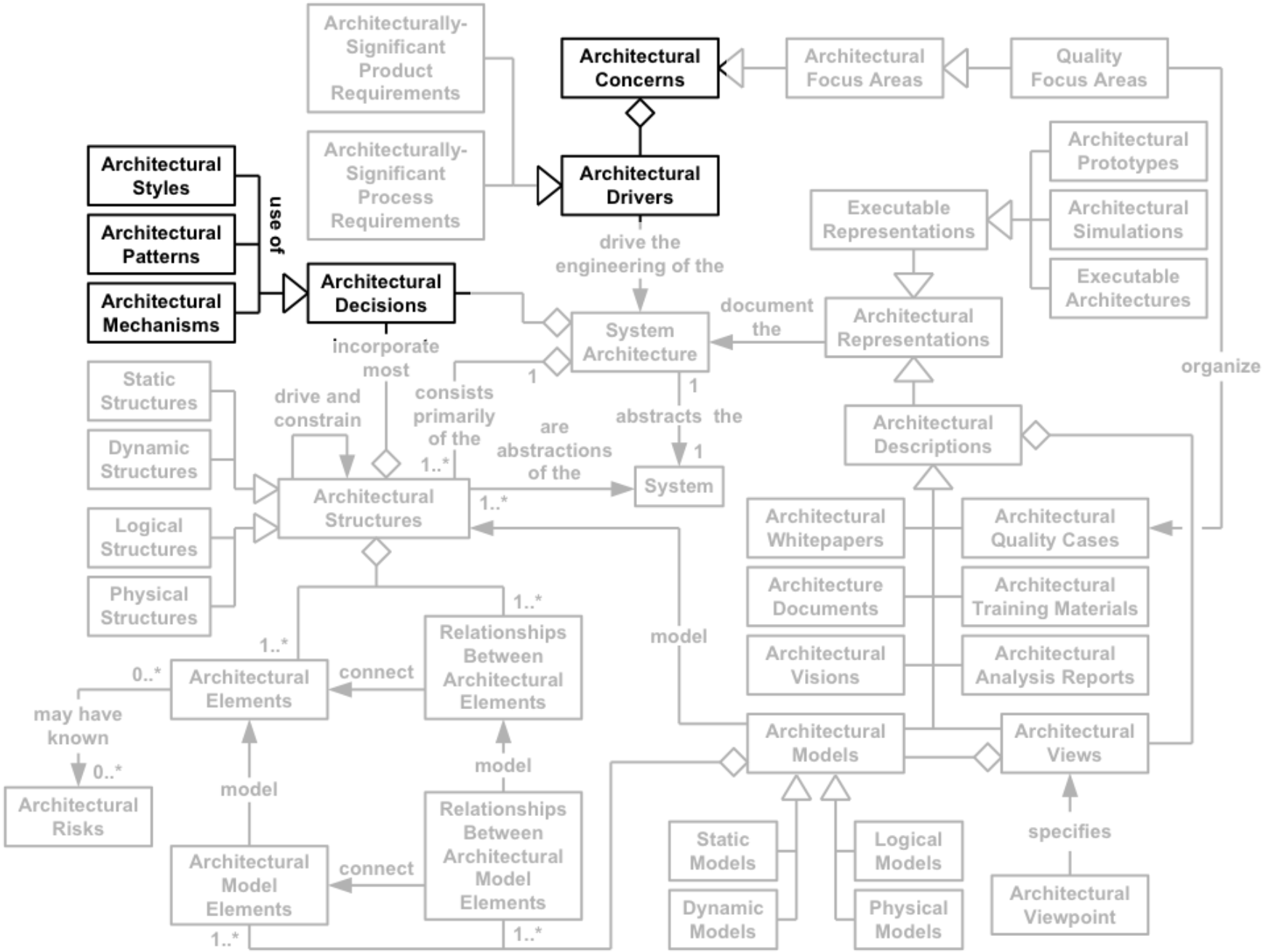
Figure 20-3. Client/server versus CQRS

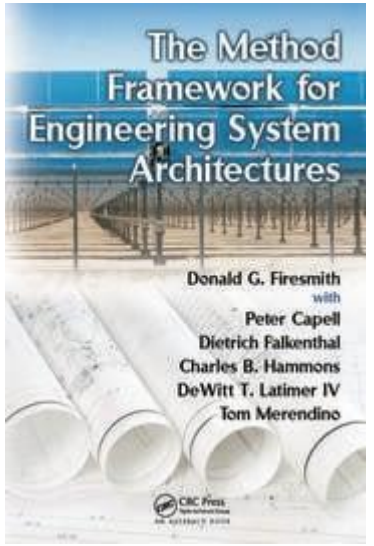




Universidad de La Sabana
FACULTAD DE INGENIERIA

How we can design ? I





[Link]

How we can design ? II

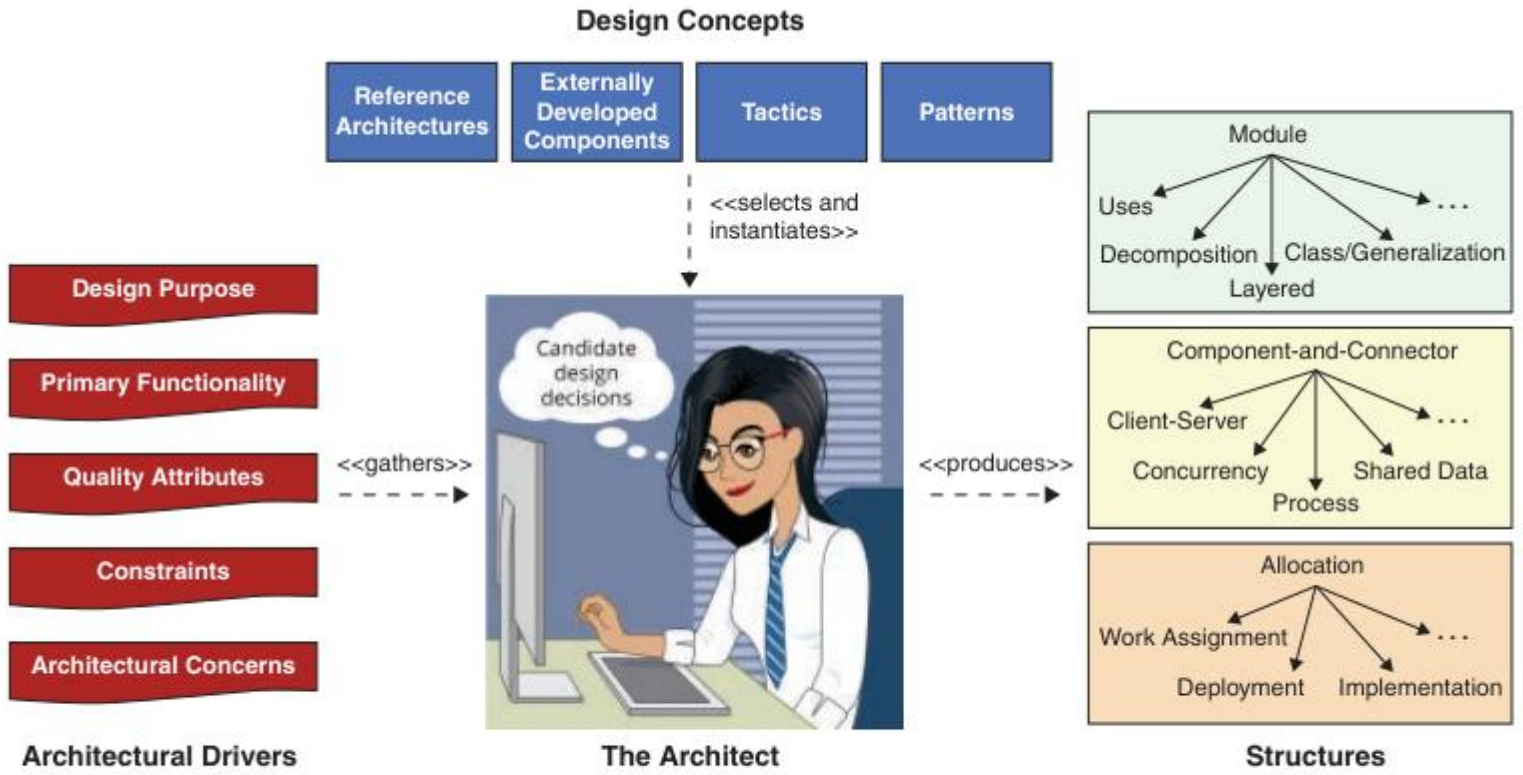
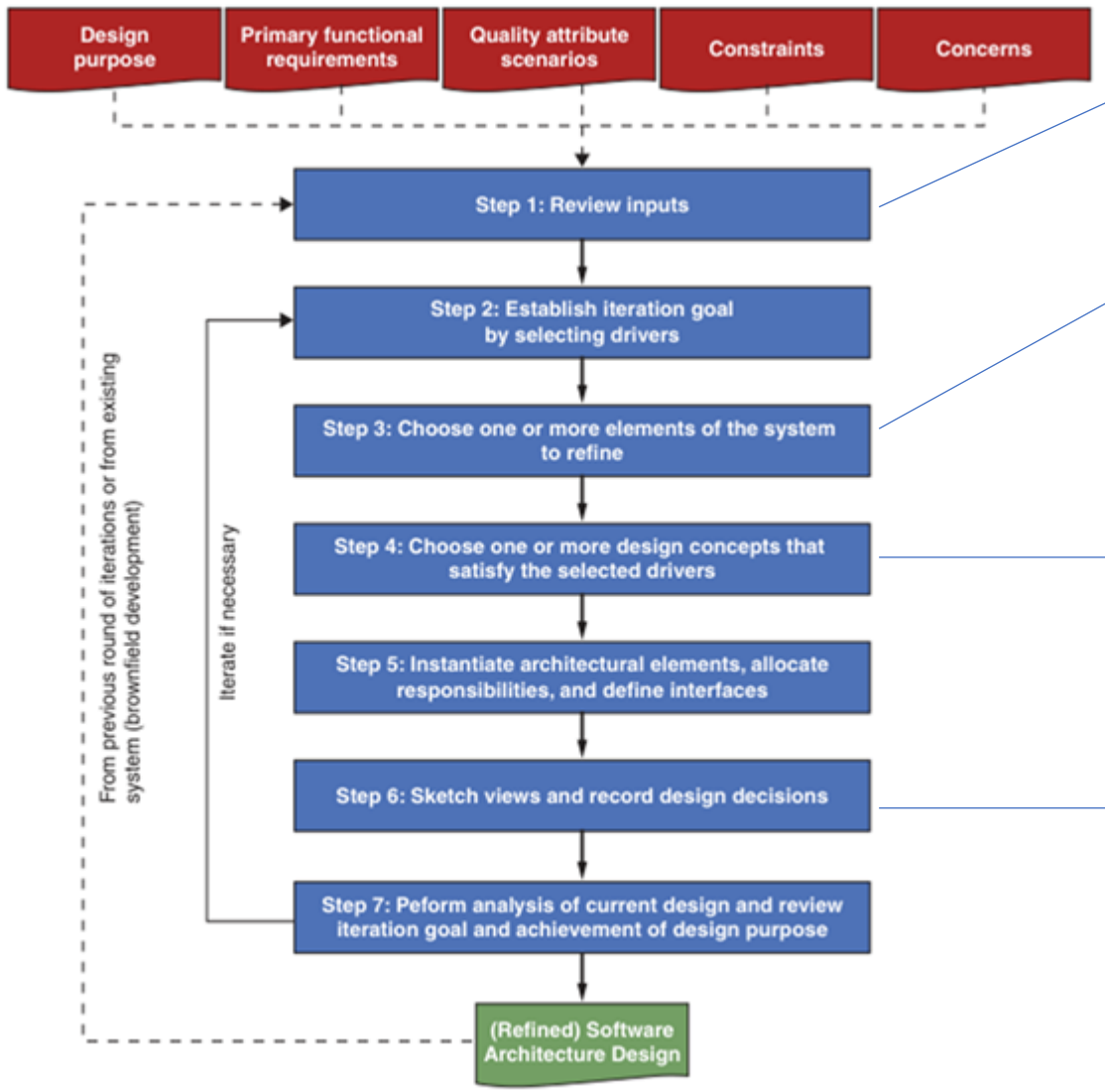


FIGURE 20.1 Overview of the architecture design activity

Attribute-Driven Design, Method



The purpose of the design round
- The primary functional requirements: User Stories
- The primary quality attribute (QA) scenarios
- Any constraints
- Any concerns

Refinement can mean decomposition into finer-grained elements (top-down approach), combination of elements into coarser-grained elements (bottom-up approach) or the improvement of previously identified elements

Choose: tactics, patterns, reference architectures, and externally developed component.
- Identification of Design Concepts: Leverage existing best practices (doc on CSPs), Leverage your own knowledge and experience (Pro: Confidence, Cons: Cognitive Bias), Leverage the knowledge and experience of others.
- Selection of Design Concepts: Pros and Cons, Decision Matrix, SWOT.
- Creation of Prototypes: Throwaway PoC.
- Value of Information: VoL.

Paper then digital format, ADR

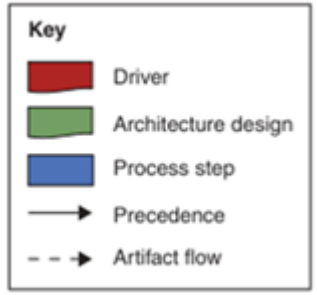


FIGURE 20.3 Steps and artifacts of ADD

Universidad de La Sabana
FACULTAD DE INGENIERIA

How to followup ?

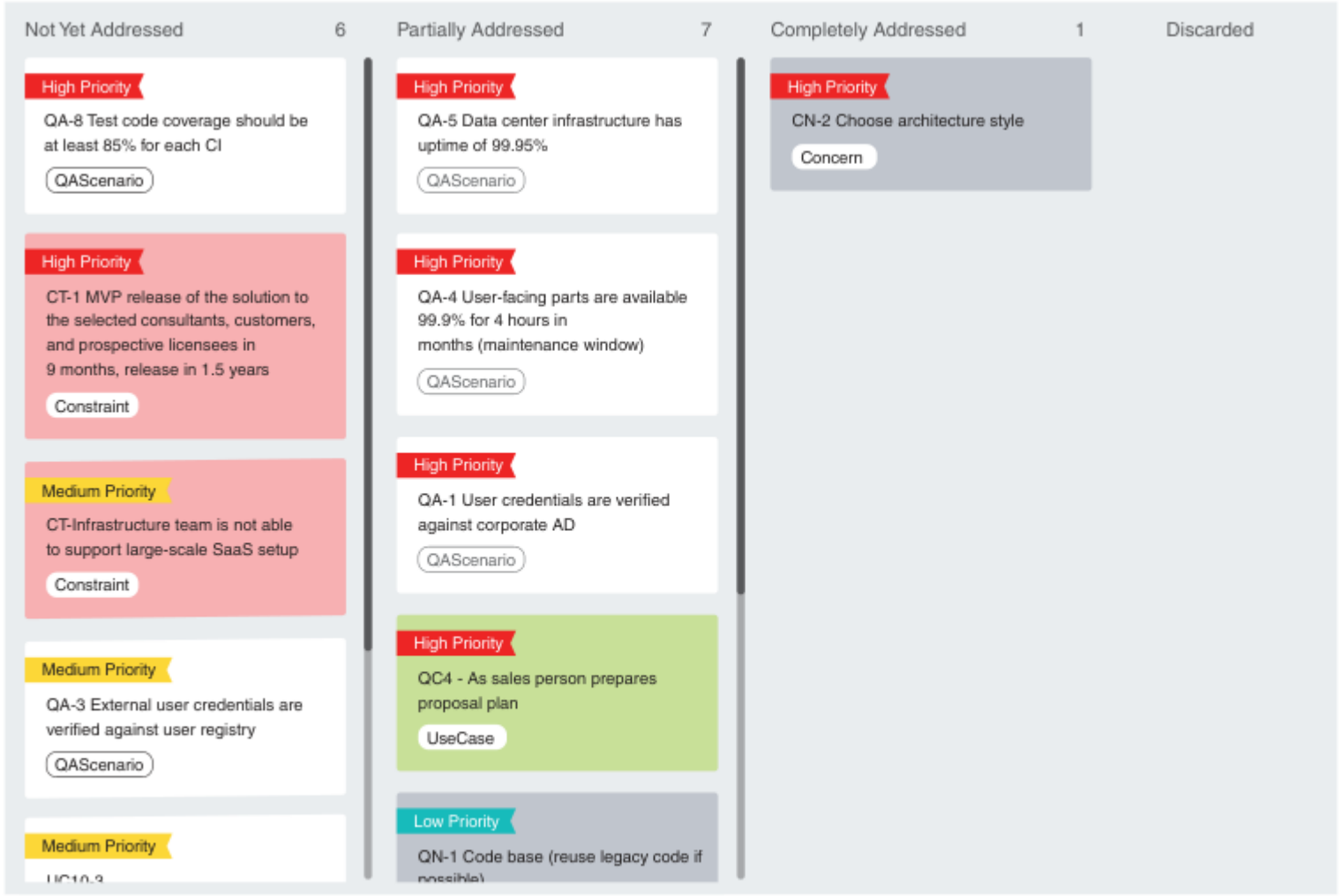


FIGURE 20.5 A Kanban board used to track design progress

Universidad de La Sabana
FACULTAD DE INGENIERIA

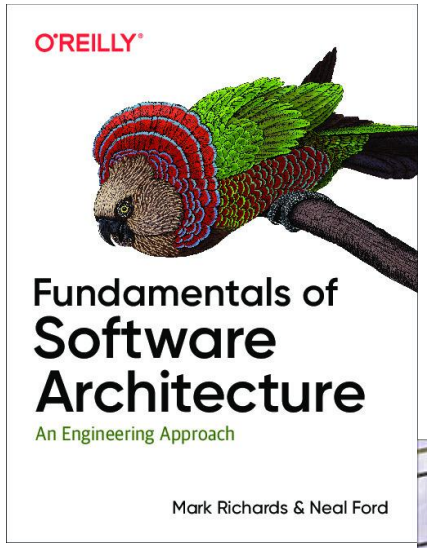
References

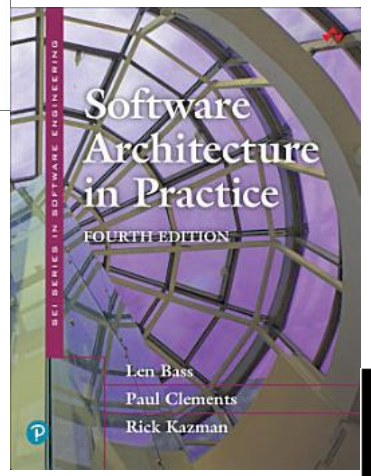
- [FoSA] Richards, M., & Ford, N. (2025). Fundamentals of software architecture (2nd ed.). O'Reilly Media.
  Ch. 1, 26 & 27

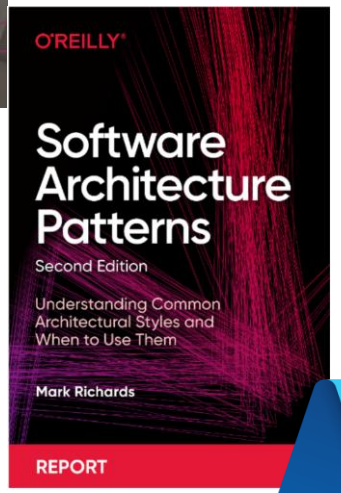
- [SAIP]Bass, L., Clements, P., & Kazman, R. (2021). Software architecture in practice (4th ed.). Addison-Wesley Professional.
  Ch. 1, {QA} 4 to 14, {ASR} 19, {ADD} 20

- [APRe] Richards, M. (2015). Software architecture patterns (Report). O'Reilly Media.
  Ch 1, {Arch Pattern} Ch. 3 to 7

- [AzDE] Microsoft. (n.d.). Architecture styles. Azure Architecture Center.



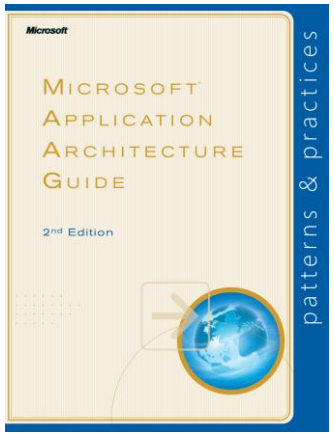


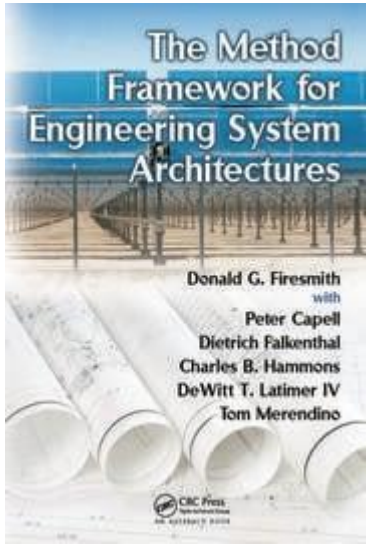


Additional References

- [MsAAG] Microsoft Application Architecture Guide (2nd Edition)
  Microsoft patterns & practices. (2009). *Microsoft application architecture guide* (2nd ed.). Microsoft Press.

- Firesmith, D. G., Capell, P., Falkenthal, D., Hammons, C. B., Latimer IV, D. T., & Merendino, T. (2008). *The method framework for engineering system architectures*. Auerbach Publications





Universidad de La Sabana

FACULTAD DE INGENIERÍA

CONTACTO

Francisco Moreno

Profesor Catedra – Facultad de Ingeniería

franciscomodi@unisabana.edu.co